



COURSE SLIDES

AI Vibe Coding with PyTorch Deep Learning

Course Code: C539

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Version v1.0 · 20 August 2026

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg



COURSE ADMINISTRATION

Welcome & Housekeeping

About the Trainer



Your Trainer

General Trainer template —
to be completed by the trainer

NAME

TITLE / DESIGNATION

QUALIFICATIONS

AREAS OF EXPERTISE

TRAINING & INDUSTRY EXPERIENCE

CONTACT

About the Trainer



Dr. Alfred Ang

Principal Trainer
Tertiary Infotech Academy Pte. Ltd.

ROLE

Principal Trainer, Tertiary Infotech Academy Pte. Ltd.

CERTIFICATION

ACLP-certified trainer; Python, deep learning and AI-assisted software development.

DELIVERS

Professional short courses in Python, data science, machine learning and deep learning.

FOUNDER

Founder and lead instructor at Tertiary Infotech / Tertiary Courses.

Let's Know Each Other

1 Your name, organisation and role.

2 Have you used Python, NumPy or PyTorch before — and have you used an AI coding assistant?

3 What images, sensor readings or sequences at work would you most like a model for?

Ground Rules

1

Set your mobile phone to silent mode.

2

Participate actively — no question is too small.

3

Mutual respect: agree to disagree.

4

One conversation at a time.

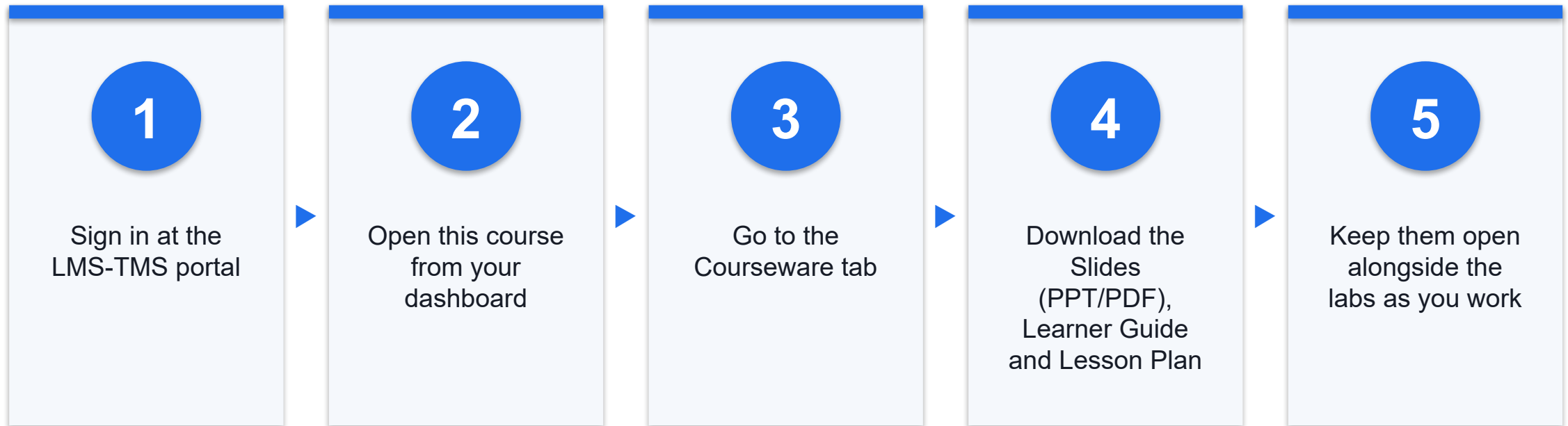
5

Be punctual; return from breaks on time.

6

Learn by doing — try every hands-on lab.

Download Course Material



Portal: <https://lms-tms.tertiaryinfotech.com>

Lesson Plan — 2 Days, 7.5 hours/day

Day 1

- Day 1 — Fundamentals to first networks - set up your AI pair programmer, vibe code tensors and autograd, then build, train and save regression and classification models.
 - Topic 1: AI Vibe Coding for PyTorch Fundamentals (Labs 1–6)
 - Topic 2: Vibe Coding Neural Networks (Labs 7–11)

Day 2

- Day 2 — Vision and sequences - vibe code a CNN image classifier, beat overfitting with augmentation and transfer learning, then forecast with an LSTM and package the project.
 - Topic 3: Vibe Coding Convolutional Neural Networks (Labs 12–16)
 - Topic 4: Vibe Coding Recurrent Networks for Sequence Data (Labs 17–21)
- Daily timing
 - 9:30am-6:30pm · 1-hour lunch · tea breaks within training time

Learning Outcomes

1

Vibe coding setup

Explain what AI vibe coding is, set up Cursor, GitHub Copilot and Claude as PyTorch coding partners, and apply prompting patterns that produce correct deep learning code.

2

Tensors and autograd

Vibe code PyTorch tensor operations, computation graphs and autograd, and review and debug AI-generated PyTorch code rather than trusting it.

3

Neural networks

Build regression and classification neural networks in PyTorch from prompts, choosing architectures, activation functions and loss functions deliberately.

4

Train and iterate

Generate training loops, optimizers and metrics from prompts, and save, load and iterate on trained models.

5

CNNs for vision

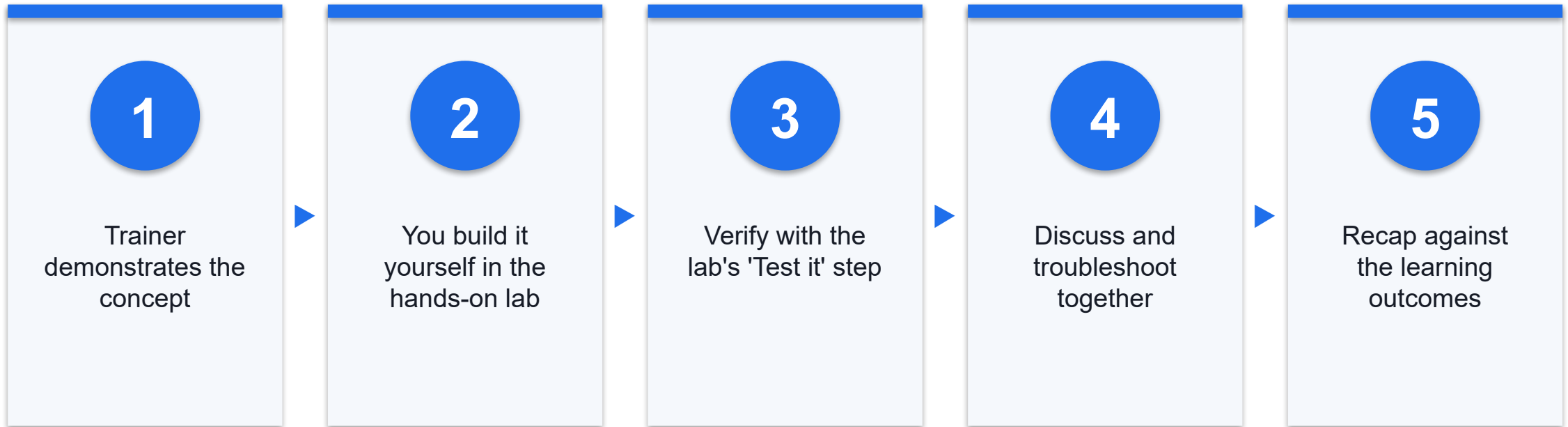
Vibe code convolutional neural networks for image classification, diagnose overfitting, and apply data augmentation, regularization and transfer learning.

6

Sequences and shipping

Vibe code recurrent networks for sequence data, tune and evaluate them with follow-up prompts, and package a complete deep learning project.

How You'll Learn





CORE CONCEPTS

Deep Learning and Vibe Coding

Key Concepts

1

Deep learning

Stacked layers that learn their own features from raw data - images, sequences, signals - instead of features you hand-engineer.

2

PyTorch

The define-by-run deep learning framework: tensors, autograd, nn.Module and an explicit training loop you can read.

3

Vibe coding

You describe the outcome in plain English; the AI assistant writes the PyTorch. You review, run, verify and correct it.

4

AI pair programmer

Cursor, GitHub Copilot or Claude inside your editor - completing, explaining, debugging and refactoring code as you work.

5

Autograd

The recorded computation graph that makes `loss.backward()` possible, and the reason a detached tensor silently stops learning.

6

Verification

The step that separates vibe coding from guessing: you check the shape, the gradient and the metric before you believe the result.

The Vibe Coding Loop

1

1. Frame

State the tensor shapes, the task, the layers, the constraints and the expected output in one clear prompt.

2

2. Generate

Let the assistant write the PyTorch. Read it before you run it and predict every shape it will print.

3

3. Run

Execute it on the real data. Look at the actual shapes, losses and predictions, not just the absence of a traceback.

4

4. Verify

Check the result against what you expected. Compare to the baseline. Inspect the samples it gets wrong.

5

5. Refine

Feed the assistant the specific symptom - not 'fix it' - and ask for a targeted change. Repeat.

6

6. Keep

Save the working script with a comment recording WHY the architecture, the loss and the metric are what they are.

THE POINT OF THIS COURSE

The AI writes the PyTorch. You own the model.

An assistant that produces runnable deep learning code is easy. An assistant that produces a model you can defend needs you to know what a right answer looks like - that is what these two days build.

What You'll Build — the ForgeSight Project

Day 1 · Fundamentals to first networks

- PyTorch, an editor and an AI assistant, verified
- A prompt pattern library that produces correct code
- Tensor and autograd utilities you wrote by prompting
- A tool-wear regression network
- A QC classifier with logits and cross entropy

Day 2 · Vision

- A convolution and pooling shape explorer
- A CNN classifier for surface-defect images
- An overfitting diagnosis you solve yourself
- An augmentation and regularization pipeline
- A fine-tuned pre-trained ResNet

Day 2 · Sequences and shipping

- An RNN, LSTM and GRU comparison
- An LSTM vibration forecaster beating persistence
- A tuned model from follow-up prompts
- Evaluation plots and a model card
- A packaged project you can hand to a colleague

How Every Lab Progresses

1

The trainer demonstrates the concept on the real ForgeSight data so you know what a correct result looks like.

2

You write the PROMPT yourself — the lab gives you a starting prompt, you adapt it to your own project folder.

3

The AI assistant generates the PyTorch; you read it and predict every shape before you run it.

4

You run it, look at the shapes and the losses, and compare them against the expected result stated in the lab.

5

The 'Test it' step forces a concrete check — a shape, a gradient, a metric, a saved file — not just 'no errors'.

6

When it is wrong, you feed the assistant the specific symptom and refine, which is the real skill.

TOPIC 01

AI Vibe Coding for PyTorch Fundamentals

What is AI vibe coding · Setting up Cursor, GitHub Copilot and Claude for PyTorch · Prompting patterns for correct deep learning code · Tensor operations · Computation graphs and autograd · Reviewing and debugging AI-generated PyTorch code

Key Concepts — AI Vibe Coding for PyTorch Fundamentals

1

Vibe coding means you describe the OUTCOME in plain language and the AI assistant writes the PyTorch. You keep the architecture, the loss, the shapes and the verification.

2

Cursor, GitHub Copilot and Claude are AI pair programmers: they complete code inline, explain an unfamiliar torch argument, and refactor a working script on request.

3

Deep learning code fails silently. A wrong loss still decreases, a wrong axis still returns a number, and a detached graph still trains to nothing, so 'it ran' is never the test.

4

A good PyTorch prompt names five things: the tensor shapes, the task, the layers, the constraints and the expected output. Vague prompts return code that runs but learns nothing.

5

A tensor is an n-dimensional array with a dtype, a device and optionally a gradient history. Almost every PyTorch bug is a shape, a dtype or a device bug.

6

Autograd records every operation on a `requires_grad` tensor into a computation graph, then replays it backwards. `loss.backward()` fills `.grad`; the optimizer consumes it.

Key Concepts — AI Vibe Coding for PyTorch Fundamentals (cont.)

7

Read the generated code before you run it and predict the printed shapes. Predicting wrongly is the fastest way to find the bug the assistant just wrote for you.

Hands-On Labs — AI Vibe Coding for PyTorch Fundamentals

Labs 1–2

- What Is AI Vibe Coding — Your First PyTorch from a Prompt
- Setting Up Cursor, GitHub Copilot and Claude for PyTorch

Labs 3–4

- Prompting Patterns for Correct Deep Learning Code
- Vibe Coding PyTorch Tensor Operations

Labs 5–6

- Computation Graphs and Autograd with AI Assistance
- Reviewing and Debugging AI-Generated PyTorch Code

What Is AI Vibe Coding — Your First PyTorch from a Prompt

LAB 1

Meet the vibe coding loop before you install anything. You open Google Colab, where PyTorch is already available, and write a PROMPT instead of a line of code. The assistant returns a script that builds a tensor of ForgeSight machine readings, prints its shape and dtype, and computes a column mean. You read that code and predict every number it will print BEFORE you run it. Then you repeat the exercise with a deliberately vague prompt and compare what comes back. The gap between the two answers is the whole lesson: the assistant is only as precise as your framing.

You'll build

A working `hello_torch.py` generated entirely from prompts, plus the first two entries in your `prompts.md` pattern library

Tools: Google Colab, PyTorch, Claude / ChatGPT / Copilot Chat

What Is AI Vibe Coding — Your First PyTorch from a Prompt

STEP 1 OF 7



1

Open Google Colab in your browser and start a new notebook. Nothing is installed today — Colab already has PyTorch, so the environment cannot be the thing that goes wrong.

What Is AI Vibe Coding — Your First PyTorch from a Prompt

STEP 2 OF 7

2

Confirm PyTorch is present and note the version. Run this in the first Colab cell.

PROMPT — paste this into your AI coding assistant

```
import torch; print(torch.__version__); print(torch.cuda.is_available())
```

What Is AI Vibe Coding — Your First PyTorch from a Prompt

STEP 3 OF 7

3

Write the VAGUE prompt first, in your AI assistant, and paste the result into a Colab cell. Do not fix anything yet — run it exactly as returned.

PROMPT — paste this into your AI coding assistant

Write some PyTorch code that works with machine sensor data.

What Is AI Vibe Coding — Your First PyTorch from a Prompt

STEP 4 OF 7

4

Now write the SPECIFIC prompt. Notice it names the shape, the dtype, the operation and the exact expected output.

PROMPT — paste this into your AI coding assistant

Write a short PyTorch script called `hello_torch.py` for a factory dataset.

It should:

1. Create a tensor of shape (6, 3) of float32 values representing 6 machine readings of (spindle_speed, coolant_temp, vibration_rms)
2. Print the tensor's shape, dtype and device on separate labelled lines
3. Print the mean of each of the 3 columns, labelled with the column name
4. Print the single reading with the highest vibration_rms
5. Use a fixed manual seed of 42 so the numbers are reproducible

Do not use pandas or numpy - torch only.

What Is AI Vibe Coding — Your First PyTorch from a Prompt

STEP 5 OF 7



5

Read the generated code line by line BEFORE running it. Write down, on paper, the shape it will print and how many numbers the column-mean line will produce.

What Is AI Vibe Coding — Your First PyTorch from a Prompt

STEP 6 OF 7



6

Run the specific version in Colab and compare the output against your written prediction.

What Is AI Vibe Coding — Your First PyTorch from a Prompt

STEP 7 OF 7



7

Start your prompt pattern library. Create prompts.md and paste BOTH prompts into it with a one-line note on what the specific one named that the vague one did not.

What Is AI Vibe Coding — Your First PyTorch from a Prompt

✓ Test it

Your specific prompt produced a script that prints `torch.Size([6, 3])`, a `float32` dtype, `cpu` as the device, `THREE` column means (not one), and one highest-vibration reading. Your written prediction of the shape matches what actually printed, and `prompts.md` contains both prompts with a note on the difference.

Setting Up Cursor, GitHub Copilot and Claude for PyTorch

LAB 2

Build the workspace every remaining lab runs in. You create the torch-vibe/ folder and an isolated virtual environment, install PyTorch, torchvision, pandas and matplotlib, then open the folder in PyCharm, VS Code or Cursor and confirm your AI assistant is actually active and able to read your files. You finish by having the assistant generate check_env.py — a script that prints every version and ends with a single 'environment ready' line, so a broken install is obvious now rather than at 4pm tomorrow in the middle of the transfer-learning lab.

You'll build

A torch-vibe/ workspace with a virtual environment, PyTorch installed and verified, and an AI assistant that can see the project

Tools: Python 3.10+, PyTorch, torchvision, PyCharm / VS Code / Cursor, GitHub Copilot / Claude

Setting Up Cursor, GitHub Copilot and Claude for PyTorch

STEP 1 OF 7

1

Create the course workspace and an isolated virtual environment so today's installs cannot break your other Python projects.

COMMAND — run this in your terminal

```
$ mkdir torch-vibe  
$ cd torch-vibe  
$ python -m venv .venv  
$ .venv\Scripts\activate
```

Setting Up Cursor, GitHub Copilot and Claude for PyTorch

STEP 2 OF 7

2

Install the CPU build of PyTorch plus the libraries every lab uses. This is the largest download of the course.

COMMAND — run this in your terminal

```
$ pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu  
$ pip install pandas matplotlib scikit-learn
```


Setting Up Cursor, GitHub Copilot and Claude for PyTorch

STEP 3 OF 7

3

Create the folder structure the whole ForgeSight project will use, and copy in the course data files.

COMMAND — run this in your terminal

```
$ mkdir data models reports  
$ cd ..
```

Setting Up Cursor, GitHub Copilot and Claude for PyTorch

STEP 4 OF 7

4

Open the workspace in your editor and confirm your AI assistant is active — you should see inline suggestions or a chat panel that can reference your open files.

COMMAND — run this in your terminal

```
$ code torch-vibe
```

Setting Up Cursor, GitHub Copilot and Claude for PyTorch

STEP 5 OF 7

5

Prompt the assistant to write the environment check. Notice you are describing the OUTPUT you want, not the imports.

PROMPT — paste this into your AI coding assistant

Create a file called `check_env.py` in this project. It should:

1. Print the Python version, then the installed versions of `torch`, `torchvision`, `pandas`, `matplotlib` and `sklearn`, one per labelled line
2. Print whether a CUDA GPU is available, and print which device the course will use ('cuda' if available else 'cpu')
3. Create a random tensor of shape (2, 3), move it to that device, and print its shape and device to prove the device line works
4. Wrap each import in a try/except that names the missing package clearly if it fails
5. Print exactly 'environment ready' as the last line if everything succeeded

Setting Up Cursor, GitHub Copilot and Claude for PyTorch

STEP 6 OF 7

6

Read the generated code, then run it. Every version should print and the last line must read 'environment ready'.

COMMAND — run this in your terminal

```
$ python check_env.py
```

Setting Up Cursor, GitHub Copilot and Claude for PyTorch

STEP 7 OF 7

7

Freeze the exact versions you installed, so the project is reproducible on another machine later in Lab 21.

COMMAND — run this in your terminal

```
$ pip freeze > requirements.txt
```

Setting Up Cursor, GitHub Copilot and Claude for PyTorch

Test it

``python check_env.py`` prints a version number for Python, torch, torchvision, pandas, matplotlib and sklearn, prints the device as 'cpu' (or 'cuda' if you have a GPU), prints a tensor of shape `torch.Size([2, 3])` on that device, and ends with exactly the line 'environment ready'. `requirements.txt` exists and contains a pinned torch version.

Prompting Patterns for Correct Deep Learning Code

LAB 3

Turn yesterday's lucky prompt into a method. You formalise the five-part pattern — shapes, task, layers and operations, constraints, expected output — and then test it under pressure on a genuinely shape-sensitive task: loading the ForgeSight telemetry and reshaping it into batches. You run a controlled A/B: the same task prompted vaguely and prompted with the pattern, and you diff the two results. You then practise the follow-up prompt, which is the skill that actually saves time — feeding back a specific symptom instead of the word 'fix'.

You'll build

A prompts.md pattern library with the five-part template and a worked A/B comparison, plus a working load_data.py

Tools: PyTorch, pandas, Cursor / GitHub Copilot / Claude

Prompting Patterns for Correct Deep Learning Code

STEP 1 OF 7



1

Write the five-part pattern into prompts.md as a reusable template you will fill in for every later lab.

Prompting Patterns for Correct Deep Learning Code

STEP 2 OF 7

2

Prompt the task VAGUELY first, and keep the result in a scratch file. You are building evidence, not code.

PROMPT — paste this into your AI coding assistant

Load the machine data and get it ready for a PyTorch model.

Prompting Patterns for Correct Deep Learning Code

STEP 3 OF 7

3

Now prompt the SAME task using the five-part pattern. Every clause maps to one part of the template.

PROMPT — paste this into your AI coding assistant

Create load_data.py for a PyTorch project.

SHAPES: data/machines.csv has 4000 rows and these columns - machine_id, spindle_speed, feed_rate, coolant_temp, vibration_rms, spindle_load, ambient_temp, tool_wear_um, qc_class.

TASK: load the CSV and return PyTorch tensors ready for supervised learning.

OPERATIONS: drop machine_id; use the 6 sensor columns as features X; return tool_wear_um as a float32 regression target and qc_class as an int64 classification target; split 70/15/15 into train/val/test with a fixed seed of 42.

CONSTRAINTS: standardise the features using the TRAINING split's mean and std only, then apply those same statistics to val and test - do not fit on the full dataset. Return the mean and std so they can be reused later.

EXPECTED OUTPUT: a function load_forgesight(path) returning a dict with keys X_train, y_wear_train, y_qc_train and the same for val and test, plus feat_mean and feat_std. When run as a script it prints the shape and dtype of every tensor.

Prompting Patterns for Correct Deep Learning Code

STEP 4 OF 7



4

Read the generated code and check ONE thing specifically: where is the mean and standard deviation computed? Find that line before you run anything.

Prompting Patterns for Correct Deep Learning Code

STEP 5 OF 7

5

Run it and confirm every shape and dtype is what the prompt asked for.

COMMAND — run this in your terminal

```
$ python load_data.py
```

Prompting Patterns for Correct Deep Learning Code

STEP 6 OF 7

6

Practise the follow-up prompt. Feed back a precise symptom rather than asking it to 'fix it', using this template with whatever your script actually got wrong.

PROMPT — paste this into your AI coding assistant

In load_data.py the features are standardised using the mean and std of the whole dataset before the split, which leaks validation and test information into training. Recompute mean and std from X_train only, then apply those same values to val and test. Change only the standardisation block.

Prompting Patterns for Correct Deep Learning Code

STEP 7 OF 7



Append both prompts and the corrected result to prompts.md, with a note naming the specific defect the pattern caught.

Prompting Patterns for Correct Deep Learning Code

Test it

`python load\_data.py` prints `X_train torch.Size([2800, 6]) float32, y_qc_train int64`, and matching val/test shapes. You can point at the exact line where mean and std are computed and confirm it uses `X_train` only. `prompts.md` holds the five-part template, both A/B prompts and the follow-up prompt.

Vibe Coding PyTorch Tensor Operations

LAB 4

Tensors are where almost every PyTorch bug actually lives, so you meet them deliberately. You prompt the assistant for a tensor workout script that runs over the real ForgeSight telemetry: dtype and device, reshape versus view, broadcasting, reduction along a chosen axis, boolean masking and matrix multiplication. Each section prints a shape, and each shape you predict first. You then trigger two shape errors on purpose and practise reading the message — because the error text tells you exactly which axis disagreed, if you know how to read it.

You'll build

A `tensor_ops.py` workout over the real telemetry, plus a `shape_errors.py` that demonstrates and explains two classic shape failures

Tools: PyTorch, pandas, Cursor / GitHub Copilot / Claude

Vibe Coding PyTorch Tensor Operations

STEP 1 OF 6

1

Prompt for the tensor workout. Note that every section is required to PRINT a shape — you cannot verify what you cannot see.

PROMPT — paste this into your AI coding assistant

Create tensor_ops.py using the load_forgesight function from load_data.py.
SHAPES: X_train is a float32 tensor of shape (2800, 6); y_wear_train is float32 of shape (2800,).
TASK: demonstrate the core tensor operations on this real data, printing the shape and dtype after every step.
OPERATIONS: (1) print shape, dtype, device and number of elements of X_train; (2) reshape y_wear_train to a column vector (2800, 1) and explain in a comment why a regression target usually needs this; (3) show the difference between .view() and .reshape() on a non-contiguous tensor; (4) compute the per-feature mean with dim=0 and the per-sample mean with dim=1 and print both shapes; (5) broadcast-subtract the per-feature mean from X_train and print the resulting shape; (6) use a boolean mask to select the rows where vibration_rms (column index 3) is above its mean, and print how many rows survived; (7) matrix-multiply X_train by a random weight tensor of shape (6, 1) and print the output shape.
CONSTRAINTS: torch only, manual seed 42, no training and no gradients.
EXPECTED OUTPUT: labelled print lines for every step above.

Vibe Coding PyTorch Tensor Operations

STEP 2 OF 6



2

Before running, write down your predicted answer to three questions: what shape does `dim=0` mean give, what shape does `dim=1` give, and what shape comes out of the `matmul`?

Vibe Coding PyTorch Tensor Operations

STEP 3 OF 6



3

Run it and check your three predictions against the output.

COMMAND — run this in your terminal

```
$ python tensor_ops.py
```

Vibe Coding PyTorch Tensor Operations

STEP 4 OF 6

4

Now break it deliberately. Prompt for a script that causes two classic shape errors and explains each one.

PROMPT — paste this into your AI coding assistant

Create `shape_errors.py` that deliberately triggers two common PyTorch shape errors and catches each one.

ERROR 1: matrix-multiply a tensor of shape (2800, 6) by a tensor of shape (1, 6) so the inner dimensions do not match.

ERROR 2: compute `MSELoss` between a prediction of shape (32, 1) and a target of shape (32,), which does NOT raise but silently broadcasts to a (32, 32) result and returns a wrong loss.

For each: wrap it in `try/except`, print the full error message for error 1, and for error 2 print the shape of the broadcast result and the wrong loss value next to the correct loss when the target is reshaped to (32, 1).

Add a comment under each explaining which axis disagreed and the one-line fix.

Vibe Coding PyTorch Tensor Operations

STEP 5 OF 6

5

Run it and read Error 1's message carefully. Identify in the text exactly which two numbers had to match and did not.

COMMAND — run this in your terminal

```
$ python shape_errors.py
```

Vibe Coding PyTorch Tensor Operations

STEP 6 OF 6



6

Study Error 2 — it is the dangerous one, because nothing raises. Confirm the two loss numbers differ and note which one is correct.

Vibe Coding PyTorch Tensor Operations

✓ Test it

tensor_ops.py prints torch.Size([6]) for the dim=0 mean, torch.Size([2800]) for the dim=1 mean and torch.Size([2800, 1]) for the matmul, and your written predictions match. shape_errors.py prints a caught error naming the (2800x6 and 1x6) mismatch, and shows a (32, 32) broadcast result whose loss value visibly differs from the correct (32, 1) loss.

Computation Graphs and Autograd with AI Assistance

LAB 5

Autograd is the machinery that makes every later lab possible, so you verify it rather than trust it. You prompt for a script that builds a tiny expression on a `requires_grad` tensor, calls `backward()`, and prints the gradient next to the value you derive analytically on paper — they must match to several decimal places. You then investigate the three ways gradients silently stop flowing: `detach()`, a `no_grad()` block, and a tensor created without `requires_grad`. Finally you watch gradients ACCUMULATE across two backward calls, which is exactly the bug a missing `zero_grad()` causes in a training loop.

You'll build

An `autograd_lab.py` proving a hand-derived gradient, plus a demonstration of `detach`, `no_grad` and gradient accumulation

Tools: PyTorch autograd, Cursor / GitHub Copilot / Claude

Computation Graphs and Autograd with AI Assistance

STEP 1 OF 7



1

Derive the gradient on paper FIRST. For $y = 3x^2 + 2x + 1$ at $x = 4$, write down dy/dx by hand before you write any prompt.

Computation Graphs and Autograd with AI Assistance

STEP 2 OF 7

2

Prompt for the verification script. You are asking the assistant to prove your arithmetic, not to teach you calculus.

PROMPT — paste this into your AI coding assistant

Create autograd_lab.py demonstrating PyTorch autograd.

PART 1 - verify a gradient: create `x = torch.tensor(4.0, requires_grad=True)`, compute `y = 3*x**2 + 2*x + 1`, call `y.backward()`, and print `x.grad`. Also print the analytic value $6x + 2$ computed manually, and assert the two agree to 5 decimal places.

PART 2 - inspect the graph: print `y.requires_grad`, `y.grad_fn` and `x.is_leaf`, with a comment explaining what each one tells you.

PART 3 - three ways gradients stop: (a) repeat Part 1 but call `.detach()` on `x` before the expression, (b) repeat it inside a `torch.no_grad()` block, (c) repeat it with `requires_grad` left as `False`. For each, print whether `grad_fn` exists and whether `x.grad` is `None`, and add a comment on when you would WANT each behaviour.

PART 4 - accumulation: with a fresh `x`, call `backward()` twice on the same expression without zeroing, printing `x.grad` after each call. Then show that `x.grad.zero_()` between calls gives the correct value both times.

CONSTRAINTS: torch only, no training loop, no `nn.Module`. Label every printed line.

Computation Graphs and Autograd with AI Assistance

STEP 3 OF 7

3

Read the code, then run Part 1 and compare the printed gradient against your paper answer.

COMMAND — run this in your terminal

```
$ python autograd_lab.py
```

Computation Graphs and Autograd with AI Assistance

STEP 4 OF 7



4

Study Part 2's output. Note which tensor has a `grad_fn` and which is a leaf — this is the shape of the computation graph in miniature.

Computation Graphs and Autograd with AI Assistance

STEP 5 OF 7



5

Work through Part 3 and answer for yourself: which of the three would you actually use at evaluation time, and which one is a bug when it appears in a model?

Computation Graphs and Autograd with AI Assistance

STEP 6 OF 7



6

Study Part 4 closely. Note the exact value printed after the second `backward()` call and compare it to the first.

Computation Graphs and Autograd with AI Assistance

STEP 7 OF 7



Add the accumulation finding to prompts.md as a checklist item for reviewing any AI-generated training loop.

Computation Graphs and Autograd with AI Assistance

Test it

Part 1 prints `x.grad` as 26.0 and the assertion against your hand-derived $6x+2$ passes. Part 2 shows `y` has a `grad_fn` while `x` is a leaf. Part 3 shows `grad_fn` absent and `x.grad` `None` in all three cases. Part 4 prints 26.0 after the first backward and 52.0 after the second, then 26.0 both times once `zero_()` is called.

Reviewing and Debugging AI-Generated PyTorch Code

LAB 6

Everything so far has been about producing code. This lab is about refusing to trust it. You are given `train_wear_buggy.py` — a script that runs to completion, prints a decreasing loss and reports believable-looking numbers, while containing five real defects. Nothing crashes. You review it against a written checklist, predict what each defect does to the result, then fix them one at a time with targeted follow-up prompts, re-running after each fix so you can see which defect was costing what. This is the single most transferable skill in the course.

You'll build

A corrected `train_wear.py`, a `review_notes.md` recording all five defects, and a reusable AI PyTorch review checklist

Tools: PyTorch, Cursor / GitHub Copilot / Claude

Reviewing and Debugging AI-Generated PyTorch Code

STEP 1 OF 7

1

Copy the buggy script from the course resources into your workspace and run it BEFORE reading it. Note the final loss and score it reports.

COMMAND — run this in your terminal

```
$ python train_wear_buggy.py
```

Reviewing and Debugging AI-Generated PyTorch Code

STEP 2 OF 7



2

Now read it against the checklist without running anything. Write your suspicions into `review_notes.md` before you ask the assistant anything.

Reviewing and Debugging AI-Generated PyTorch Code

STEP 3 OF 7

3

Ask the assistant to review it — but constrain the review so it reports rather than rewrites.

PROMPT — paste this into your AI coding assistant

Review the PyTorch script `train_wear_buggy.py` for correctness defects. Do NOT rewrite the file. For each defect, report exactly three things: (1) the line, quoted verbatim; (2) what it does to the RESULT - be specific about whether it makes the score too high, too low, or meaningless; (3) the minimal one-line fix. Check specifically for: standardisation or scaling fitted before the train/test split; a missing or misplaced `optimizer.zero_grad()`; a softmax or sigmoid applied before a loss function that already applies it; evaluation performed without `model.eval()` or `torch.no_grad()`; a target tensor whose shape does not match the prediction shape. Rank the defects by how much they distort the reported result.

Reviewing and Debugging AI-Generated PyTorch Code

STEP 4 OF 7



4

Compare the assistant's list against your own. Anything you found that it missed is worth more than anything it found that you missed — record both in `review_notes.md`.

Reviewing and Debugging AI-Generated PyTorch Code

STEP 5 OF 7

5

Fix the defects ONE AT A TIME, re-running after each. Start with the one the assistant ranked most distorting.

PROMPT — paste this into your AI coding assistant

In `train_wear_buggy.py`, the features are standardised using the mean and standard deviation of the full dataset before the train/test split, which leaks test information into training. Recompute the mean and std from the training split only and apply those same values to the test split. Change only that block and save the result as `train_wear.py`.

Reviewing and Debugging AI-Generated PyTorch Code

STEP 6 OF 7



6

Continue with the remaining defects, using the same specific-symptom style. After each fix, record the new loss and score in `review_notes.md`.

Reviewing and Debugging AI-Generated PyTorch Code

STEP 7 OF 7



7

Compare the original reported numbers against the corrected ones, and write one sentence in `review_notes.md` explaining why the buggy script's numbers looked believable while both models were actually stuck at their baselines.

Reviewing and Debugging AI-Generated PyTorch Code

Test it

review_notes.md lists all five defects with the line quoted, the effect on the result and the fix. train_wear.py runs clean, and its corrected tool-wear MAE is clearly BETTER than the buggy script's — which was no better than predicting the average — while the corrected QC accuracy rises above the share of 'ok' parts the buggy version was merely echoing. You can state in one sentence why the buggy numbers looked believable, and your reusable checklist is written down in prompts.md.

Recap — AI Vibe Coding for PyTorch Fundamentals

1

You can now: explain what AI vibe coding is and generate, read, run and verify your first PyTorch script from a plain-language prompt

2

You can now: set up a local Python environment with PyTorch and an AI coding assistant that can see your project files

3

You can now: apply a repeatable five-part prompting pattern that produces correct, runnable deep learning code

4

You can now: create, reshape, broadcast, index and move tensors, and read a shape error well enough to know which axis is wrong

5

You can now: explain what autograd records, verify a gradient by hand, and show what detach and no_grad actually change

6

You can now: review AI-generated PyTorch against a fixed checklist and correct each defect with a targeted follow-up prompt

TOPIC 02

Vibe Coding Neural Networks

Neural network architectures, activation and loss functions · Vibe coding a regression model · Vibe coding a classification model with softmax and cross entropy · Generating training loops, optimizers and metrics from prompts · Saving, loading and iterating on models

Key Concepts — Vibe Coding Neural Networks

1

A neural network is a stack of linear layers separated by non-linearities. Without the activation function the whole stack collapses into one linear layer.

2

ReLU is the default hidden activation; sigmoid and tanh saturate and stall learning in deep stacks. The OUTPUT layer's activation is decided by the task, not by taste.

3

The loss function must match the output layer: MSELoss for regression, CrossEntropyLoss for multi-class. Mismatch them and the model trains happily towards nonsense.

4

`nn.CrossEntropyLoss` applies log-softmax internally, so the model must output RAW LOGITS. Adding a softmax layer before it is the single most common PyTorch bug.

5

The training loop is always the same five lines: zero the gradients, forward, compute loss, backward, step. Forgetting `zero_grad()` accumulates gradients across batches.

6

Optimizers differ in how they use the gradient: SGD follows it, Adam adapts a per-parameter learning rate. Learning rate matters more than the choice of optimizer.

Key Concepts — Vibe Coding Neural Networks (cont.)

7

Save the `state_dict`, not the pickled object, and record the architecture alongside it - weights without the class that built them are unloadable.

Hands-On Labs — Vibe Coding Neural Networks

Labs 7–8

- Neural Network Architectures, Activation and Loss Functions
- Vibe Coding a Regression Model in PyTorch

Labs 9–10

- Vibe Coding a Classification Model with Softmax and Cross Entropy
- Generating Training Loops, Optimizers and Metrics from Prompts

Lab 11

- Saving, Loading and Iterating on Models

Neural Network Architectures, Activation and Loss Functions

LAB 7

Before you train anything, settle the three decisions that determine whether training can work at all: the architecture, the activation and the loss. You prompt for a script that plots ReLU, sigmoid and tanh next to their gradients, then proves the claim that a stack of linear layers without activations collapses into a single linear layer — by showing two networks produce identical outputs. You finish by building a decision table linking each task type to its output layer and its loss, which becomes the reference you use for the next fourteen labs.

You'll build

An `activations.py` with plotted activations and a linear-collapse proof, plus a `losses_cheatsheet.md` decision table

Tools: PyTorch, matplotlib, Cursor / GitHub Copilot / Claude

Neural Network Architectures, Activation and Loss Functions

STEP 1 OF 7

1

Prompt for the activation comparison. Ask for the gradients as well as the functions — the gradient is what explains the behaviour.

PROMPT — paste this into your AI coding assistant

Create activations.py for a PyTorch course.

TASK: compare the three common activation functions and show why the choice matters.

OPERATIONS: (1) create `x = torch.linspace(-6, 6, 200)`; (2) compute ReLU, sigmoid and tanh of `x` using `torch.nn.functional`; (3) compute the gradient of each with respect to `x` using `autograd`; (4) plot a 2-row figure - top row the three activations, bottom row their three gradients, sharing the `x` axis - and save it to `reports/activations.png`; (5) print, for each activation, the fraction of the input range where its gradient is smaller than 0.01.

CONSTRAINTS: `torch` and `matplotlib` only, no `seaborn`, label every axis and subplot.

EXPECTED OUTPUT: the saved figure plus three printed 'saturated fraction' lines.

Neural Network Architectures, Activation and Loss Functions

STEP 2 OF 7

2

Run it, open the figure, and answer one question from the bottom row: which activation keeps a usable gradient across the widest input range?

COMMAND — run this in your terminal

```
$ python activations.py
```

Neural Network Architectures, Activation and Loss Functions

STEP 3 OF 7

3

Now prove the linear-collapse claim. This is the reason activations exist at all.

PROMPT — paste this into your AI coding assistant

Add a section to `activations.py` called `linear_collapse()`.

TASK: prove that a stack of linear layers with no activation is equivalent to a single linear layer.

OPERATIONS: (1) build `net_a = nn.Sequential(nn.Linear(6, 32), nn.Linear(32, 16), nn.Linear(16, 1))` with manual seed 42; (2) analytically collapse its three weight matrices and biases into ONE equivalent `nn.Linear(6, 1)` by composing them; (3) run the same random input batch of shape (8, 6) through both and print the two outputs side by side plus their maximum absolute difference; (4) assert the difference is below $1e-4$; (5) repeat with `nn.ReLU()` inserted between the layers and print the maximum absolute difference now.

EXPECTED OUTPUT: near-zero difference without activations, clearly non-zero difference with ReLU.

Neural Network Architectures, Activation and Loss Functions

STEP 4 OF 7

4

Run it and confirm the assertion passes. Read the second difference — that number is what the non-linearity is buying you.

COMMAND — run this in your terminal

```
$ python activations.py
```

Neural Network Architectures, Activation and Loss Functions

STEP 5 OF 7



5

Build the decision table by hand in `losses_cheatsheet.md`. Fill it in yourself before checking it with the assistant.

Neural Network Architectures, Activation and Loss Functions

STEP 6 OF 7

6

Check your table against the assistant, and specifically interrogate the classification row.

PROMPT — paste this into your AI coding assistant

I have written a table mapping task type to output layer and PyTorch loss function:

- Regression (one continuous value) -> Linear(h, 1), no activation -> nn.MSELoss
- Binary classification -> Linear(h, 1), no activation -> nn.BCEWithLogitsLoss
- Multi-class classification (4 classes) -> Linear(h, 4), no activation -> nn.CrossEntropyLoss

For each row, confirm or correct it, and explain in one sentence what happens numerically if someone adds a softmax or sigmoid to the output layer before these losses. Be specific about which losses already apply that function internally.

Neural Network Architectures, Activation and Loss Functions

STEP 7 OF 7



Record the answer in `losses_cheatsheet.md` in your own words, especially the softmax warning.

Neural Network Architectures, Activation and Loss Functions

Test it

reports/activations.png shows three activations over three gradients, and the printed saturated fractions are near zero for ReLU and clearly non-zero for sigmoid and tanh. `linear_collapse()` asserts a difference below $1e-4$ without activations and prints a visibly larger difference with ReLU. `losses_cheatsheet.md` has all three rows and states which losses apply softmax or sigmoid internally.

Vibe Coding a Regression Model in PyTorch

LAB 8

Build ForgeSight's first real model: a network predicting tool wear in microns from six sensor readings. You prompt for an `nn.Module` with a deliberate architecture, an `MSELoss`, and a training loop that reports train and validation loss each epoch. Crucially you establish a BASELINE first — predicting the training mean for every row — because a regression score means nothing on its own. You then check the prediction and target shapes explicitly, which is where the (N,) versus (N,1) trap from Lab 4 comes back to bite anyone who skipped it.

You'll build

A trained tool-wear regression network reporting MAE and RMSE that beat the mean baseline

Tools: PyTorch, `nn.Module`, `MSELoss`, Cursor / GitHub Copilot / Claude

Vibe Coding a Regression Model in PyTorch

STEP 1 OF 7

1

Compute the baseline FIRST, before any model exists. You cannot judge a regression score without it.

PROMPT — paste this into your AI coding assistant

Create `baseline_wear.py`. Using `load_forgesight` from `load_data.py`, predict the MEAN of `y_wear_train` for every row in the validation set, and print the resulting MAE and RMSE in microns. Print the standard deviation of `y_wear_train` alongside them. Add a comment explaining why RMSE for a mean-predictor is close to the target's standard deviation.

Vibe Coding a Regression Model in PyTorch

STEP 2 OF 7

2

Run it and write the baseline MAE down. Every number you produce for the rest of this lab is judged against it.

COMMAND — run this in your terminal

```
$ python baseline_wear.py
```

Vibe Coding a Regression Model in PyTorch

STEP 3 OF 7

3

Prompt for the model and the training script. Note that the prompt names the shapes, the loss and the exact reporting format.

PROMPT — paste this into your AI coding assistant

Create models.py and train_wear.py for a PyTorch regression task.
SHAPES: X_train is float32 (2800, 6), y_wear_train is float32 (2800,). Features are already standardised from load_data.py.
TASK: predict tool_wear_um, a continuous value in microns.
OPERATIONS: in models.py define class WearNet(nn.Module) with Linear(6,64) -> ReLU -> Linear(64,32) -> ReLU -> Linear(32,1) and NO activation on the output. In train_wear.py use nn.MSELoss and torch.optim.Adam at lr=1e-3, batch size 64 via TensorDataset and DataLoader, and train for 100 epochs.
CONSTRAINTS: reshape the target to (N,1) so it matches the prediction shape exactly - print both shapes once before the first loss call to prove they match. Set manual seed 42. Evaluate under model.eval() and torch.no_grad(). Do not apply any activation to the output layer.
EXPECTED OUTPUT: per-epoch train and validation loss every 10 epochs; final validation MAE and RMSE in microns; a saved plot of both loss curves at reports/wear_curve.png.

Vibe Coding a Regression Model in PyTorch

STEP 4 OF 7



4

Read the generated code and check three things before running: the output layer has no activation, the target is reshaped, and `eval()` plus `no_grad()` wrap the validation pass.

Vibe Coding a Regression Model in PyTorch

STEP 5 OF 7

5

Run the training. Watch that both losses fall and that the printed shapes match.

COMMAND — run this in your terminal

```
$ python train_wear.py
```

Vibe Coding a Regression Model in PyTorch

STEP 6 OF 7



6

Compare your final validation MAE against the baseline MAE you wrote down. State the improvement as a percentage.

Vibe Coding a Regression Model in PyTorch

STEP 7 OF 7



7

Open reports/wear_curve.png and read the two curves. Note whether validation loss is still falling at epoch 100 or has flattened.

Vibe Coding a Regression Model in PyTorch

✓ Test it

baseline_wear.py prints a baseline MAE and RMSE. train_wear.py prints matching prediction and target shapes of `torch.Size([N, 1])`, trains with both losses falling, and reports a final validation MAE clearly **BELOW** the baseline MAE. reports/wear_curve.png shows both curves labelled.

Vibe Coding a Classification Model with Softmax and Cross Entropy

LAB 9

Predict the ForgeSight quality-control class — ok, scratch, dent or burr — from the same six sensors. This lab is deliberately a trap. You first ask the assistant for the model in a way that invites the classic mistake, and there is a strong chance it hands you a network with `nn.Softmax` on the output next to `nn.CrossEntropyLoss`. You catch it, prove numerically that it is wrong, and fix it. You then evaluate properly against a majority-class baseline with a confusion matrix, because on imbalanced classes accuracy alone will flatter a model that never predicts the rare defect at all.

You'll build

A QC classifier outputting raw logits, evaluated with accuracy, per-class recall and a confusion matrix against the majority baseline

Tools: PyTorch, `nn.CrossEntropyLoss`, scikit-learn metrics, Cursor / GitHub Copilot / Claude

Vibe Coding a Classification Model with Softmax and Cross Entropy

STEP 1 OF 8

1

Establish the majority-class baseline first, and look at how imbalanced the classes actually are.

PROMPT — paste this into your AI coding assistant

Create `baseline_qc.py`. Using `load_forgesight`, print the count and percentage of each of the 4 `qc_class` values in the training split. Then predict the single most frequent class for every validation row and print the resulting accuracy. Print the per-class recall for that baseline as well, and add a comment on what recall is for the classes it never predicts.

Vibe Coding a Classification Model with Softmax and Cross Entropy

STEP 2 OF 8

2

Run it. Note the majority-class accuracy — this is the number your model must beat, and it is higher than most people expect.

COMMAND — run this in your terminal

```
$ python baseline_qc.py
```

Vibe Coding a Classification Model with Softmax and Cross Entropy

STEP 3 OF 8

3

Now prompt for the classifier, deliberately **WITHOUT** specifying the output activation. You are setting a trap for the assistant.

PROMPT — paste this into your AI coding assistant

Add class QCNet(nn.Module) to models.py and create train_qc.py.
SHAPES: X_train is float32 (2800, 6); y_qc_train is int64 (2800,) with 4 classes.
TASK: classify each machine reading into one of 4 quality-control classes.
OPERATIONS: QCNet should be Linear(6,64) -> ReLU -> Linear(64,32) -> ReLU -> Linear(32,4). Train with nn.CrossEntropyLoss and Adam at lr=1e-3, batch size 64, 100 epochs.
CONSTRAINTS: manual seed 42, evaluate under model.eval() and torch.no_grad().
EXPECTED OUTPUT: per-epoch train and validation loss, final validation accuracy, per-class precision and recall, and a confusion matrix saved to reports/qc_confusion.png.

Vibe Coding a Classification Model with Softmax and Cross Entropy

STEP 4 OF 8



4

STOP before running. Inspect the generated QCNet output layer. Is there an `nn.Softmax`, `nn.LogSoftmax` or `F.softmax` anywhere after the final Linear?

Vibe Coding a Classification Model with Softmax and Cross Entropy

STEP 5 OF 8

5

If a softmax is present, prove it is wrong before removing it rather than just deleting it.

PROMPT — paste this into your AI coding assistant

Write a short script `proof_softmax.py` that takes one batch of logits of shape (8, 4) with manual seed 42 and computes `nn.CrossEntropyLoss` twice: once on the raw logits, and once on `torch.softmax(logits, dim=1)`. Print both loss values and both gradients with respect to the logits, and print the ratio of the gradient magnitudes. Add a comment explaining that `CrossEntropyLoss` applies `log_softmax` internally, so the second version applies softmax twice and produces much smaller gradients.

Vibe Coding a Classification Model with Softmax and Cross Entropy

STEP 6 OF 8

6

Fix the model with a targeted follow-up, then train it.

PROMPT — paste this into your AI coding assistant

In models.py, QCNet applies softmax to its output, but nn.CrossEntropyLoss already applies log_softmax internally, so the network is applying it twice and its gradients are being flattened. Remove the softmax layer so QCNet returns raw logits from the final Linear layer. Keep everything else unchanged. Where a probability is needed for reporting, apply torch.softmax at that point instead.

Vibe Coding a Classification Model with Softmax and Cross Entropy

STEP 7 OF 8

7

Train the corrected model and compare accuracy and per-class recall against the majority baseline.

COMMAND — run this in your terminal

```
$ python train_qc.py
```


Vibe Coding a Classification Model with Softmax and Cross Entropy

STEP 8 OF 8



8

Open `reports/qc_confusion.png` and find which class the model confuses most. Note whether it predicts the rarest class at all.

Vibe Coding a Classification Model with Softmax and Cross Entropy

Test it

`baseline_qc.py` prints class counts and the majority-class accuracy. `proof_softmax.py` prints two different loss values and shows the double-softmax gradients are much smaller. `train_qc.py`'s QCNet returns raw logits with no softmax layer, and reports a validation accuracy above the majority baseline with a confusion matrix saved to `reports/qc_confusion.png`.

Generating Training Loops, Optimizers and Metrics from Prompts

LAB 10

You have now written the same training loop twice. Refactor it once, properly, and never write it again. You prompt for `engine.py` containing a single `fit()` that takes a model, loaders, a loss, an optimizer and a metric function, returns a history dictionary, and works unchanged for both the regression and the classification task. With that in place, experimentation becomes cheap: you run a small sweep comparing SGD against Adam across three learning rates and produce a results table that shows learning rate matters more than the choice of optimizer.

You'll build

A reusable `engine.py` fit/evaluate pair driving both models, plus a sweep table comparing optimizers and learning rates

Tools: PyTorch, `torch.optim`, `DataLoader`, Cursor / GitHub Copilot / Claude

Generating Training Loops, Optimizers and Metrics from Prompts

STEP 1 OF 7

1

Prompt for the reusable engine. The hard requirement is that it must work for BOTH tasks without modification.

PROMPT — paste this into your AI coding assistant

Create engine.py with two functions that work unchanged for both a regression and a classification task. fit(model, train_loader, val_loader, loss_fn, optimizer, epochs, metric_fn=None, device='cpu') should:

1. Loop over epochs; for each batch call optimizer.zero_grad() BEFORE the forward pass, compute the loss, call backward, then step
2. Accumulate the training loss weighted by batch size, not a plain mean of batch means
3. After each epoch run a validation pass under model.eval() and torch.no_grad(), then return the model to train mode
4. Apply metric_fn(preds, targets) on the validation set if one is given
5. Return a history dict with keys train_loss, val_loss and val_metric, each a list of length epochs
6. Track and restore the state_dict from the epoch with the best validation loss, and report which epoch that was

evaluate(model, loader, loss_fn, metric_fn, device) should run one pass under eval/no_grad and return the loss and metric.

CONSTRAINTS: no printing inside fit except an optional every-N-epochs line controlled by a verbose argument; no task-specific logic - the caller supplies loss_fn and metric_fn.

EXPECTED OUTPUT: engine.py importable by both train_wear.py and train_qc.py.

Generating Training Loops, Optimizers and Metrics from Prompts

STEP 2 OF 7



2

Review the generated `fit()` against your Lab 6 checklist before you trust it — this function is about to run every experiment for the rest of the course.

Generating Training Loops, Optimizers and Metrics from Prompts

STEP 3 OF 7

3

Rewire both existing training scripts to use it, and confirm the results still match what you got in Labs 8 and 9.

PROMPT — paste this into your AI coding assistant

Refactor `train_wear.py` and `train_qc.py` to use `fit` and `evaluate` from `engine.py` instead of their own inline training loops. Keep the same seeds, architectures, hyperparameters and reporting so the results are directly comparable. `train_wear.py` passes `nn.MSELoss` and an MAE metric function; `train_qc.py` passes `nn.CrossEntropyLoss` and an accuracy metric function. Delete the now-duplicated loop code from both files.

Generating Training Loops, Optimizers and Metrics from Prompts

STEP 4 OF 7

4

Re-run both and check the final numbers are essentially unchanged from the previous labs. A refactor that changes results is a refactor that introduced a bug.

COMMAND — run this in your terminal

```
$ python train_wear.py  
$ python train_qc.py
```

Generating Training Loops, Optimizers and Metrics from Prompts

STEP 5 OF 7

5

Now use the engine for what it was built for. Prompt for a sweep across optimizers and learning rates.

PROMPT — paste this into your AI coding assistant

Create sweep.py using fit from engine.py.

TASK: compare optimizers and learning rates on the QC classification task.

OPERATIONS: for each combination of optimizer in [SGD with momentum 0.9, Adam] and learning rate in [1e-2, 1e-3, 1e-4], train a FRESH QCNet for 60 epochs with manual seed 42 reset before each run, and record the best validation loss, the validation accuracy at that epoch, the epoch it occurred, and the wall-clock seconds.

CONSTRAINTS: identical data, batch size and seed across all 6 runs - only the optimizer and learning rate change. Re-instantiate the model each run so no weights carry over.

EXPECTED OUTPUT: a printed table sorted by best validation loss, saved to reports/sweep.csv, plus a single figure overlaying all 6 validation loss curves with a legend.

Generating Training Loops, Optimizers and Metrics from Prompts

STEP 6 OF 7

6

Run the sweep and read the table. Answer: does the optimizer or the learning rate account for more of the spread in results?

COMMAND — run this in your terminal

```
$ python sweep.py
```

Generating Training Loops, Optimizers and Metrics from Prompts

STEP 7 OF 7



Record the winning configuration and your answer in prompts.md, and adopt that configuration for the rest of the course.

Generating Training Loops, Optimizers and Metrics from Prompts

Test it

engine.py's fit runs both train_wear.py and train_qc.py unchanged and reproduces the Labs 8 and 9 results. sweep.py produces reports/sweep.csv with 6 rows, a sorted printed table, and an overlay figure of 6 validation curves. You can state which factor - optimizer or learning rate - drove more of the difference.

Saving, Loading and Iterating on Models

LAB 11

A model that only exists in memory is not a deliverable. You prompt for a checkpoint helper that saves the `state_dict` together with everything needed to rebuild it — architecture arguments, the standardisation statistics, the class names, the metrics and the library versions — then reload it into a freshly constructed model in a NEW process and assert the predictions match to machine precision. That assertion is the difference between believing your model saved and knowing it did. You finish by writing the first ForgeSight model card.

You'll build

A checkpoint helper saving weights plus config and statistics, a proven identical-prediction reload, and a model card

Tools: PyTorch, `torch.save` / `torch.load`, Cursor / GitHub Copilot / Claude

Saving, Loading and Iterating on Models

STEP 1 OF 7

1

Prompt for the checkpoint helper. The key insight is that the file must contain everything needed to rebuild, not just the weights.

PROMPT — paste this into your AI coding assistant

Create checkpoint.py with save_checkpoint and load_checkpoint functions.

save_checkpoint(path, model, config, feat_mean, feat_std, metrics, class_names=None) must save a single dict containing: the model's state_dict; the config dict of constructor arguments needed to rebuild the class; the model class NAME as a string; feat_mean and feat_std tensors; the metrics dict; the class names; the torch version; and an ISO timestamp.

load_checkpoint(path, model_class) must construct a fresh instance from the saved config, load the state_dict into it, call model.eval(), and return the model plus the full metadata dict.

CONSTRAINTS: save the state_dict, never the pickled model object - explain why in a comment. Create the models/ directory if missing. Use weights_only=False on load only where required by your torch version, and note why in a comment.

EXPECTED OUTPUT: both functions importable, with a __main__ block that round-trips a small dummy model as a self-test.

Saving, Loading and Iterating on Models

STEP 2 OF 7

2

Wire it into the QC training script so a completed training run always leaves a loadable artifact behind.

PROMPT — paste this into your AI coding assistant

Update `train_qc.py` to call `save_checkpoint` at the end of training. Save to `models/qc_net_v1.pt` with the config needed to rebuild QCNet, the `feat_mean` and `feat_std` returned by `load_forgesight`, the final validation accuracy and per-class recall as metrics, and `class_names=['ok','scratch','dent','burr']`.

Saving, Loading and Iterating on Models

STEP 3 OF 7



3

Run training so the checkpoint is written.

COMMAND — run this in your terminal

```
$ python train_qc.py
```

Saving, Loading and Iterating on Models

STEP 4 OF 7

4

Now prove the reload works, from a SEPARATE script that never sees the trained object in memory.

PROMPT — paste this into your AI coding assistant

Create `reload_check.py` that proves the checkpoint round-trips exactly.

OPERATIONS: (1) load `models/qc_net_v1.pt` with `load_checkpoint`, rebuilding QCNet from the saved config; (2) take the first 16 rows of the validation set; (3) run them through the reloaded model under `torch.no_grad()`; (4) print the predicted class indices and the saved metrics; (5) assert the reloaded model's predictions are identical to the predictions the training script produced for the same rows, which you should also save alongside the checkpoint for this purpose; (6) print the saved timestamp, torch version and class names.

CONSTRAINTS: this script must NOT import or re-run training - it may only read the checkpoint file.

Saving, Loading and Iterating on Models

STEP 5 OF 7

5

Run it and confirm the assertion passes. This is the moment you know the artifact is real.

COMMAND — run this in your terminal

```
$ python reload_check.py
```

Saving, Loading and Iterating on Models

STEP 6 OF 7

6

Deliberately break it to see the failure mode you are guarding against.

PROMPT — paste this into your AI coding assistant

Add a commented-out demonstration to `reload_check.py`: reload the model but skip applying `feat_mean` and `feat_std` to the input rows, so the model receives raw unstandardised sensor values. Print the predictions from both paths side by side and a comment noting that no error is raised - the model simply predicts confidently and wrongly.

Saving, Loading and Iterating on Models

STEP 7 OF 7



Write the model card. Do it in your own words — this is the document that travels with the model.

Saving, Loading and Iterating on Models

✓ Test it

models/qc_net_v1.pt exists and contains the state_dict, config, feat_mean, feat_std, metrics, class names, torch version and timestamp. reload_check.py runs in a fresh process, rebuilds QCNet from the checkpoint alone, and its assertion of identical predictions passes. models/qc_model_card.md states the metric, the baseline and at least one place the model must not be used.

Recap — Vibe Coding Neural Networks

1

You can now: choose hidden and output activations and match the loss function to the output layer

2

You can now: build, train and honestly evaluate a regression network that predicts a continuous value

3

You can now: build a multi-class classifier that outputs raw logits and pair it correctly with cross entropy loss

4

You can now: refactor training into one reusable fit function and compare optimizers and learning rates with it

5

You can now: save a model's `state_dict` with its configuration, reload it into a fresh instance and prove the predictions are identical

TOPIC 03

Vibe Coding Convolutional Neural Networks

Overview of CNNs: convolution, pooling and padding · Vibe coding a CNN image classifier · Diagnosing overfitting with AI assistance · Data augmentation and regularization via prompts · Transfer learning with pre-trained models

Key Concepts — Vibe Coding Convolutional Neural Networks

1

A convolution slides a small learned kernel across the image, so the same feature detector works wherever the feature appears - this is why CNNs beat dense layers on images.

2

Padding preserves spatial size at the border; stride and pooling shrink it. Getting these wrong is how the flatten-to-linear layer ends up with a shape mismatch.

3

Pooling downsamples and buys a little translation tolerance. Channels grow as spatial size shrinks: the network trades WHERE for WHAT as it goes deeper.

4

Overfitting is the model memorising training images. You see it as a widening gap between training and validation loss, not as a bad training number.

5

Data augmentation - random flips, rotations, crops, colour jitter - manufactures new training views for free. Augment the TRAINING set only, never validation.

6

Dropout, weight decay and early stopping are the three regularizers you reach for first. Dropout must be switched off at evaluation time by `model.eval()`.

Key Concepts — Vibe Coding Convolutional Neural Networks (cont.)

7

Transfer learning reuses a network already trained on millions of images: freeze the backbone, replace the classifier head, and fine-tune on a few hundred of your own images.

Hands-On Labs — Vibe Coding Convolutional Neural Networks

Labs 12–13

- Overview of CNNs: Convolution, Pooling and Padding
- Vibe Coding a CNN Image Classifier

Labs 14–15

- Diagnosing Overfitting with AI Assistance
- Data Augmentation and Regularization via Prompts

Lab 16

- Transfer Learning with Pre-Trained Models

Overview of CNNs: Convolution, Pooling and Padding

LAB 12

ForgeSight gains a camera. You generate the surface-inspection image set — 1200 greyscale images of machined parts labelled ok, scratch or dent — then build a shape explorer rather than a model. You apply Conv2d and MaxPool2d with different kernel sizes, paddings and strides, predicting each output shape from the formula before you print it. You visualise what an edge-detecting kernel actually does to a scratch image, and you finish by deriving the flatten size that a classifier's first Linear layer needs — the number that causes more CNN shape errors than anything else.

You'll build

A generated 1200-image defect dataset plus a `conv_explorer.py` that predicts and verifies every feature-map shape

Tools: PyTorch, torchvision, Pillow, matplotlib, Cursor / GitHub Copilot / Claude

Overview of CNNs: Convolution, Pooling and Padding

STEP 1 OF 8

1

Generate the ForgeSight surface-inspection images. Copy `make_images.py` from the course resources and run it.

COMMAND — run this in your terminal

```
$ python make_images.py
```

Overview of CNNs: Convolution, Pooling and Padding

STEP 2 OF 8

2

Inspect what you generated before you model it. Never train on a dataset you have not looked at.

PROMPT — paste this into your AI coding assistant

Create `peek_images.py` that loads the generated dataset from `data/defects/` and: (1) prints how many images are in each class folder for both train and val; (2) loads one image and prints its size, mode and the shape it becomes as a tensor via `torchvision.transforms.ToTensor()`; (3) saves a 3x4 grid figure to `reports/sample_defects.png` showing four examples of each of the three classes with the class name as the subplot title.

CONSTRAINTS: use PIL and torchvision only. Do not train anything.

Overview of CNNs: Convolution, Pooling and Padding

STEP 3 OF 8

3

Run it, open `reports/sample_defects.png`, and describe out loud what visually distinguishes a scratch from a dent. If you cannot see it, the model will struggle too.

COMMAND — run this in your terminal

```
$ python peek_images.py
```

Overview of CNNs: Convolution, Pooling and Padding

STEP 4 OF 8



4

Now the shape work. Write down the output-size formula and predict three shapes on paper before prompting.

Overview of CNNs: Convolution, Pooling and Padding

STEP 5 OF 8

5

Prompt for the explorer, which forces you to commit to a prediction before it reveals the answer.

PROMPT — paste this into your AI coding assistant

Create conv_explorer.py demonstrating how convolution and pooling change tensor shape.
SHAPES: the input is a batch of 8 greyscale images of shape (8, 1, 64, 64).
TASK: show the output shape of each operation and confirm it against the analytic formula.
OPERATIONS: for each of these configurations, print the config, the analytically computed output size using the formula $\text{floor}((W - K + 2P)/S) + 1$, and the ACTUAL shape from running the layer, then assert they agree:

- (a) Conv2d(1, 8, kernel_size=3, padding=0, stride=1)
- (b) Conv2d(1, 8, kernel_size=3, padding=1, stride=1)
- (c) Conv2d(1, 8, kernel_size=5, padding=2, stride=1)
- (d) Conv2d(1, 8, kernel_size=3, padding=1, stride=2)
- (e) MaxPool2d(kernel_size=2, stride=2) applied after (b)

Then build a small stack Conv(1,8,3,p=1) -> ReLU -> MaxPool(2) -> Conv(8,16,3,p=1) -> ReLU -> MaxPool(2) and print the shape after every single layer, finishing with the number of features a Flatten would produce.
CONSTRAINTS: no training, no gradients, manual seed 42. Print a comment line explaining which configuration preserves spatial size and why.

Overview of CNNs: Convolution, Pooling and Padding

STEP 6 OF 8

6

Run it and check your three paper predictions against the printed shapes.

COMMAND — run this in your terminal

```
$ python conv_explorer.py
```


Overview of CNNs: Convolution, Pooling and Padding

STEP 7 OF 8



Visualise what a convolution actually computes, so the operation stops being abstract.

PROMPT — paste this into your AI coding assistant

Add a section to `conv_explorer.py` that takes one scratch image and one ok image from `data/defects/`, applies three FIXED 3x3 kernels - a horizontal edge detector, a vertical edge detector, and a blur - using `F.conv2d` with manually set weights rather than learned ones, and saves a figure to `reports/feature_maps.png` showing the original next to the three responses for both images. Add a comment on which kernel responds most strongly to a scratch and why.

Overview of CNNs: Convolution, Pooling and Padding

STEP 8 OF 8

8

Run it, open the figure, and note which kernel makes the scratch most visible. Record the final flatten size from the stack — you need it in Lab 13.

COMMAND — run this in your terminal

```
$ python conv_explorer.py
```

Overview of CNNs: Convolution, Pooling and Padding

Test it

data/defects/ contains train and val folders with three class subfolders each and roughly 1200 images total. conv_explorer.py prints predicted and actual shapes for all five configurations with every assertion passing, prints the full stack's per-layer shapes ending in a flatten size of 4096, and reports/feature_maps.png shows the edge kernels responding to a scratch.

Vibe Coding a CNN Image Classifier

LAB 13

Turn the shape knowledge into a working classifier. You prompt for an ImageFolder-based data pipeline and a DefectCNN whose first Linear layer takes exactly the flatten size you derived in Lab 12, then train it with the same `engine.fit()` you built in Lab 10 — no new training loop. You evaluate against a majority-class baseline with a confusion matrix, and you inspect the images the model gets wrong. Because the architecture, the loss and the loop are all things you have already validated, the only new failure surface is the data pipeline and the shape arithmetic.

You'll build

A trained DefectCNN classifying ok, scratch and dent, evaluated with a confusion matrix and a misclassified-image grid

Tools: PyTorch, torchvision ImageFolder, engine.py, Cursor / GitHub Copilot / Claude

Vibe Coding a CNN Image Classifier

STEP 1 OF 8

1

Prompt for the image data pipeline. Keep the transforms minimal for now — augmentation is deliberately Lab 15's job.

PROMPT — paste this into your AI coding assistant

Create datasets.py for the defect images.
SHAPES: data/defects/train and data/defects/val each contain subfolders ok, scratch and dent holding 64x64 greyscale PNGs.
TASK: return DataLoaders for training and validation.
OPERATIONS: build a function `get_defect_loaders(batch_size=32, data_dir='data/defects')` that uses `torchvision.datasets.ImageFolder` with a transform pipeline of `Grayscale(1)` then `ToTensor()` then `Normalize(mean=[0.5], std=[0.5])`. Return `train_loader` (shuffled), `val_loader` (not shuffled) and the `class_to_idx` mapping.
CONSTRAINTS: do NOT add any augmentation - that comes in a later lab. Set a fixed generator seed of 42 for the training shuffle. Print the `class_to_idx` mapping and the number of images in each split when run as a script.
EXPECTED OUTPUT: importable loaders plus a printed summary showing the three classes and their counts.

Vibe Coding a CNN Image Classifier

STEP 2 OF 8

2

Run it and confirm the class mapping. Note which integer maps to which class name — you will need it to read the confusion matrix.

COMMAND — run this in your terminal

```
$ python datasets.py
```

Vibe Coding a CNN Image Classifier

STEP 3 OF 8

3

Prompt for the CNN. Give it the flatten size you derived in Lab 12 rather than letting it guess.

PROMPT — paste this into your AI coding assistant

Add class DefectCNN(nn.Module) to models.py.

SHAPES: input batches are (N, 1, 64, 64); there are 3 output classes.

ARCHITECTURE: Conv2d(1,16,3,padding=1) -> ReLU -> MaxPool2d(2) -> Conv2d(16,32,3,padding=1) -> ReLU -> MaxPool2d(2) -> Conv2d(32,64,3,padding=1) -> ReLU -> MaxPool2d(2) -> Flatten -> Linear(64*8*8, 128) -> ReLU -> Linear(128, 3).

CONSTRAINTS: the final layer must output RAW LOGITS with no softmax, because training uses nn.CrossEntropyLoss. Add a comment above the Flatten deriving 64*8*8 from three poolings of a 64x64 input. Include a __main__ block that runs a random (4,1,64,64) tensor through the model and prints the shape after each block to prove the arithmetic.

Vibe Coding a CNN Image Classifier

STEP 4 OF 8

4

Before training, run the model's self-test and confirm the printed shapes match your own derivation.

COMMAND — run this in your terminal

```
$ python models.py
```


Vibe Coding a CNN Image Classifier

STEP 5 OF 8

5

Prompt for the training script, reusing the engine rather than writing another loop.

PROMPT — paste this into your AI coding assistant

Create `train_cnn.py` that trains DefectCNN using `fit` and `evaluate` from `engine.py` - do NOT write a new training loop.
OPERATIONS: get the loaders from `datasets.py`; instantiate DefectCNN with manual seed 42; train for 25 epochs with `nn.CrossEntropyLoss`, Adam at `lr=1e-3`, and an accuracy metric function; print the majority-class baseline accuracy on the validation set BEFORE training starts; after training report final validation accuracy, per-class precision and recall, and save a labelled confusion matrix to `reports/cnn_confusion.png`; save a checkpoint to `models/defect_cnn_v1.pt` using `save_checkpoint` from `checkpoint.py`; save the train and validation loss curves to `reports/cnn_curve.png`.
CONSTRAINTS: evaluation under `model.eval()` and `torch.no_grad()` - which `fit` already handles. Use the class names from `class_to_idx` on the confusion matrix axes.

Vibe Coding a CNN Image Classifier

STEP 6 OF 8

6

Train it. On CPU this takes a few minutes — watch the two loss curves as they print.

COMMAND — run this in your terminal

```
$ python train_cnn.py
```

Vibe Coding a CNN Image Classifier

STEP 7 OF 8



7

Compare final accuracy against the majority baseline, then read the confusion matrix to see which two classes the model confuses.

Vibe Coding a CNN Image Classifier

STEP 8 OF 8

8

Look at what it got wrong, which tells you more than any single number.

PROMPT — paste this into your AI coding assistant

Create `inspect_errors.py` that loads `models/defect_cnn_v1.pt`, runs the validation set, selects up to 12 misclassified images, and saves a grid to `reports/cnn_errors.png` where each subplot title shows the true class, the predicted class and the model's confidence in its wrong answer. Sort them by confidence so the most confidently wrong images appear first.

Vibe Coding a CNN Image Classifier

✓ Test it

`datasets.py` prints three classes with their counts and the `class_to_idx` mapping. `models.py`'s self-test prints per-block shapes ending in a flatten of 4096. `train_cnn.py` reports a validation accuracy clearly above the printed majority-class baseline, saves `reports/cnn_confusion.png` with named axes, and writes `models/defect_cnn_v1.pt`. `reports/cnn_errors.png` shows misclassified images with confidences.

Diagnosing Overfitting with AI Assistance

LAB 14

Your Lab 13 model almost certainly overfits — now measure it rather than guess. You train the same CNN deliberately hard and plot training against validation loss and accuracy on shared axes, then identify the exact epoch where validation stops improving while training keeps falling. To make the pattern unmistakable you also train on a deliberately tiny subset until it reaches near-perfect training accuracy and useless validation accuracy. You finish by asking the assistant to diagnose the curves from a description alone, and you check its reasoning against what you can see.

You'll build

An overfitting diagnosis with annotated curves, a memorised-subset demonstration, and a written diagnosis you verified yourself

Tools: PyTorch, matplotlib, engine.py, Cursor / GitHub Copilot / Claude

Diagnosing Overfitting with AI Assistance

STEP 1 OF 8

1

Train long enough for the problem to appear. Prompt for a diagnostic run with no regularization at all.

PROMPT — paste this into your AI coding assistant

Create `diagnose_overfit.py` that trains DefectCNN for 60 epochs using `fit` from `engine.py` with NO augmentation, NO dropout and NO weight decay - deliberately unregularized.

OPERATIONS: record train loss, validation loss, train accuracy and validation accuracy every epoch. Save a 2-panel figure to `reports/overfit_curves.png` - left panel both losses, right panel both accuracies, epochs on the x axis, with a vertical dashed line marking the epoch of MINIMUM validation loss and that epoch number in the legend. Print: the best validation loss and its epoch; the final training loss; the final validation loss; the final gap between training and validation accuracy in percentage points.

CONSTRAINTS: manual seed 42, batch size 32, Adam $\text{lr}=1\text{e-}3$. Do not stop early - run all 60 epochs so the divergence is visible.

Diagnosing Overfitting with AI Assistance

STEP 2 OF 8

2

Run it. This takes several minutes on CPU — while it runs, write down what you expect the two curves to do.

COMMAND — run this in your terminal

```
$ python diagnose_overfit.py
```


Diagnosing Overfitting with AI Assistance

STEP 3 OF 8

3

Open reports/overfit_curves.png and answer three questions before reading any explanation: at which epoch does validation loss bottom out, what does training loss do after that, and how wide is the final accuracy gap?

Diagnosing Overfitting with AI Assistance

STEP 4 OF 8

4

Now make the effect undeniable. Prompt for a memorisation demonstration on a tiny subset.

PROMPT — paste this into your AI coding assistant

Add a function `memorise_demo()` to `diagnose_overfit.py`.

TASK: show overfitting in its purest form by training on far too little data.

OPERATIONS: take only 30 training images (10 per class) using `torch.utils.data.Subset`, keep the FULL validation set, and train a fresh DefectCNN for 100 epochs with the same settings. Plot training and validation accuracy to `reports/memorise.png` and print both final accuracies.

CONSTRAINTS: same seed and architecture as the main run - only the amount of training data changes.

EXPECTED OUTPUT: training accuracy approaching 100% while validation accuracy stays near the majority-class baseline.

Diagnosing Overfitting with AI Assistance

STEP 5 OF 8

5

Run it and compare the two numbers. This is memorisation with nothing learned.

COMMAND — run this in your terminal

```
$ python diagnose_overfit.py
```

Diagnosing Overfitting with AI Assistance

STEP 6 OF 8

6

Test the assistant's diagnostic reasoning — and then test the assistant.

PROMPT — paste this into your AI coding assistant

I trained a CNN image classifier for 60 epochs with no regularization. Training loss fell steadily from 1.05 to 0.04 and training accuracy reached 99%. Validation loss fell until epoch 14, reaching 0.52, then rose steadily to 0.95 by epoch 60, while validation accuracy peaked at 78% around epoch 14 and drifted down to 71%.

Diagnose what is happening, name the epoch I should have stopped at, and rank the following fixes by how much improvement you would expect for THIS symptom, with a one-line reason each: more training data, data augmentation, dropout, weight decay, early stopping, a smaller model, a lower learning rate.

Be explicit about which of these address the cause and which only limit the damage.

Diagnosing Overfitting with AI Assistance

STEP 7 OF 8



7

Compare the assistant's ranking against your own curves. Does its recommended stopping epoch match the dashed line in your figure?

Diagnosing Overfitting with AI Assistance

STEP 8 OF 8



8

Write the diagnosis into `review_notes.md` in your own words, with the specific numbers from YOUR run.

Diagnosing Overfitting with AI Assistance

✓ Test it

reports/overfit_curves.png shows training loss falling while validation loss turns upward, with a dashed line at the minimum-validation-loss epoch and that epoch printed. reports/memorise.png shows near-100% training accuracy against near-baseline validation accuracy on 30 images. review_notes.md records your best epoch, your accuracy gap and the fixes you will apply next.

Data Augmentation and Regularization via Prompts

LAB 15

Fix what you measured. You add the four standard remedies one at a time — augmentation on the training set only, dropout in the classifier head, weight decay in the optimizer, and early stopping via the best-epoch tracking already in `engine.fit()` — and after each change you record the validation accuracy and the train/validation gap. The critical trap is deliberate: augmentation must never be applied to validation, and an assistant asked for 'augmented loaders' will very often apply it to both, which silently makes your validation score noisy and pessimistic.

You'll build

A regularized training pipeline plus an ablation table showing the measured contribution of each remedy

Tools: PyTorch, torchvision.transforms, engine.py, Cursor / GitHub Copilot / Claude

Data Augmentation and Regularization via Prompts

STEP 1 OF 8

1

Add augmented loaders, with an explicit instruction about which split gets augmented.

PROMPT — paste this into your AI coding assistant

Update datasets.py with a new function `get_augmented_loaders(batch_size=32, data_dir='data/defects')`.

TASK: return training and validation loaders where ONLY the training set is augmented.

OPERATIONS: the TRAINING transform is `RandomHorizontalFlip(0.5)`, `RandomRotation(10)`, `RandomResizedCrop(64, scale=(0.8, 1.0))`, then `Grayscale(1)`, `ToTensor()`, `Normalize([0.5],[0.5])`. The VALIDATION transform is ONLY `Grayscale(1)`, `ToTensor()`, `Normalize([0.5],[0.5])` with NO random operations whatsoever.

CONSTRAINTS: this is the critical requirement - validation must be deterministic, because a randomly augmented validation set gives a different score every run and cannot be compared across experiments. Keep the original `get_defect_loaders` unchanged so the two can be compared.

EXPECTED OUTPUT: a `__main__` block that prints the two transform pipelines side by side so the difference is visible, and saves a grid of 8 augmented versions of the SAME training image to `reports/augmented_samples.png`.

Data Augmentation and Regularization via Prompts

STEP 2 OF 8

2

Verify the trap did not catch you. Print both pipelines and confirm no random transform appears in the validation list.

COMMAND — run this in your terminal

```
$ python datasets.py
```

Data Augmentation and Regularization via Prompts

STEP 3 OF 8



3

Open reports/augmented_samples.png. Check the augmentations are plausible — a rotation so extreme that a scratch becomes unrecognisable teaches the model nothing.

Data Augmentation and Regularization via Prompts

STEP 4 OF 8

4

Add dropout to the model, in the right place.

PROMPT — paste this into your AI coding assistant

Add class DefectCNNv2(nn.Module) to models.py, identical to DefectCNN but with nn.Dropout(p=0.3) inserted after the ReLU that follows Linear(4096, 128), and nn.Dropout2d(p=0.1) after the final MaxPool.

Add a comment explaining why dropout goes in the classifier head rather than between early convolutional layers, and a second comment noting that model.eval() disables it automatically - which is why evaluation must never run in train mode.

Keep the output as raw logits with no softmax.

Data Augmentation and Regularization via Prompts

STEP 5 OF 8

5

Run the ablation. This is the actual experiment — five configurations, one change at a time.

PROMPT — paste this into your AI coding assistant

Create `train_cnn_v2.py` running an ablation over regularization, using `fit` from `engine.py`.

RUNS - each 40 epochs, fresh model, manual seed 42 reset before each:

1. baseline: DefectCNN, plain loaders, Adam $lr=1e-3$, `weight_decay=0`
2. plus augmentation: DefectCNN, augmented loaders
3. plus dropout: DefectCNNv2, augmented loaders
4. plus weight decay: DefectCNNv2, augmented loaders, `weight_decay=1e-4`
5. plus early stopping: as run 4 but reporting the metrics from the BEST validation epoch rather than the last

For each run record: best validation loss and its epoch, final validation accuracy, best validation accuracy, final training accuracy, and the train-validation accuracy gap in percentage points.

EXPECTED OUTPUT: a printed table in run order, saved to `reports/ablation.csv`, plus a figure overlaying the five validation loss curves with a legend.

Data Augmentation and Regularization via Prompts

STEP 6 OF 8

6

Run the ablation and read the gap column down the table. Which single change reduced the gap most?

COMMAND — run this in your terminal

```
$ python train_cnn_v2.py
```

Data Augmentation and Regularization via Prompts

STEP 7 OF 8



Save the best configuration as the new ForgeSight defect model.

PROMPT — paste this into your AI coding assistant

Update `train_cnn_v2.py` to retrain the winning configuration and save it with `save_checkpoint` to `models/defect_cnn_v2.pt`, including in the metrics dict the validation accuracy, the best epoch, and a regularization field listing which techniques were used. Update `models/defect_model_card.md` with the new score, the baseline for comparison, and one sentence on what changed since v1.

Data Augmentation and Regularization via Prompts

STEP 8 OF 8



8

Compare v2's accuracy and gap against the Lab 14 numbers you wrote in `review_notes.md`.

Data Augmentation and Regularization via Prompts

✓ Test it

`datasets.py` prints a validation pipeline containing no random transforms, and `reports/augmented_samples.png` shows eight plausible variants of one training image. `reports/ablation.csv` holds five rows with a train-validation gap column that narrows down the table. `models/defect_cnn_v2.pt` is saved with a regularization field, and v2's gap is smaller than the Lab 14 gap.

Transfer Learning with Pre-Trained Models

LAB 16

Stop training from scratch. You load a ResNet-18 pre-trained on ImageNet, adapt it to greyscale 64x64 inspection images, freeze the backbone so only a new three-class head learns, and fine-tune. You then unfreeze the last block at a much lower learning rate and compare all three approaches — scratch CNN, frozen backbone, partial fine-tune — on accuracy, training time and trainable parameter count. The lab makes concrete why transfer learning is the default professional starting point when you have hundreds of images rather than hundreds of thousands.

You'll build

A fine-tuned ResNet-18 defect classifier plus a three-way comparison against your scratch CNN

Tools: PyTorch, torchvision.models, engine.py, Cursor / GitHub Copilot / Claude

Transfer Learning with Pre-Trained Models

STEP 1 OF 8

1

Prompt for the transfer setup, being explicit about the two adaptations a greyscale 64x64 input requires.

PROMPT — paste this into your AI coding assistant

Create transfer.py that fine-tunes a pre-trained ResNet-18 on the defect images.

TASK: adapt an ImageNet ResNet-18 to 3-class greyscale 64x64 defect classification.

OPERATIONS: (1) load torchvision.models.resnet18 with the default pre-trained weights; (2) adapt the input - the simplest correct approach is a transform pipeline that converts the greyscale image to 3 channels and resizes to 224x224 using the ImageNet normalisation statistics, so state clearly in a comment which approach you used and why; (3) FREEZE every parameter by setting requires_grad=False; (4) replace model.fc with a new nn.Linear(512, 3) whose parameters are trainable; (5) print the total parameter count and the TRAINABLE parameter count so the difference is obvious; (6) train for 12 epochs with Adam lr=1e-3 using fit from engine.py, passing only the trainable parameters to the optimizer.

CONSTRAINTS: the new head outputs raw logits. Use the augmented training transform and the deterministic validation transform from Lab 15, adjusted for 3-channel 224x224 ImageNet input.

EXPECTED OUTPUT: printed parameter counts, per-epoch losses, final validation accuracy, and the elapsed training time.

Transfer Learning with Pre-Trained Models

STEP 2 OF 8



2

Before training, check the parameter counts. The trainable number should be a tiny fraction of the total.

COMMAND — run this in your terminal

```
$ python transfer.py
```

Transfer Learning with Pre-Trained Models

STEP 3 OF 8



3

Note the accuracy and time, then compare against your Lab 15 scratch model trained for far longer.

Transfer Learning with Pre-Trained Models

STEP 4 OF 8

4

Now unfreeze the last block and fine-tune at a much lower learning rate.

PROMPT — paste this into your AI coding assistant

Add a function `partial_finetune()` to `transfer.py`.

OPERATIONS: start from the frozen model trained above, then set `requires_grad=True` for `layer4` and `fc` only. Use two parameter groups in Adam - `layer4` at `lr=1e-4` and `fc` at `lr=1e-3` - and train for a further 8 epochs. Print the new trainable parameter count and the final validation accuracy.

Add a comment explaining why the unfrozen backbone layer uses a learning rate roughly ten times smaller than the head: the pre-trained weights are already good, and a large update would destroy the features you are trying to reuse.

CONSTRAINTS: do not re-initialise the head - continue from the weights just trained.

Transfer Learning with Pre-Trained Models

STEP 5 OF 8

5

Run the partial fine-tune and record whether the extra 8 epochs improved on the frozen result.

COMMAND — run this in your terminal

```
$ python transfer.py
```

Transfer Learning with Pre-Trained Models

STEP 6 OF 8

6

Build the three-way comparison table that answers the practical question.

PROMPT — paste this into your AI coding assistant

Add a comparison to `transfer.py` that produces `reports/transfer_comparison.csv` with one row per approach: (1) DefectCNNv2 trained from scratch with the Lab 15 best configuration; (2) ResNet-18 frozen backbone with a new head; (3) ResNet-18 with layer4 unfrozen. Columns: approach, trainable parameters, epochs trained, wall-clock training seconds, best validation accuracy, and validation accuracy per minute of training. Print the table sorted by best validation accuracy and add a one-line printed conclusion naming which approach you would choose for a factory with 1200 labelled images and no GPU.

Transfer Learning with Pre-Trained Models

STEP 7 OF 8

7

Save the best transfer model with a checkpoint and a model card entry.

PROMPT — paste this into your AI coding assistant

Update transfer.py to save the best-performing transfer model with save_checkpoint to models/defect_resnet_v1.pt, recording in the metrics dict the approach used, which layers were trainable, the validation accuracy and the training time. Append a section to models/defect_model_card.md comparing the scratch CNN and the transfer model, and stating which one you recommend for deployment and why.

Transfer Learning with Pre-Trained Models

STEP 8 OF 8



8

Read the comparison table and state your recommendation out loud, with the trade-off you are accepting.

Transfer Learning with Pre-Trained Models

Test it

transfer.py prints a trainable parameter count that is a tiny fraction of the ~11M total, trains a frozen-backbone ResNet-18 to a validation accuracy comparable with or better than your scratch CNN, and reports the partial fine-tune result. reports/transfer_comparison.csv holds three rows with parameters, time and accuracy. models/defect_resnet_v1.pt is saved and the model card names a recommendation.

Recap — Vibe Coding Convolutional Neural Networks

1

You can now: reason about convolution, padding, stride and pooling well enough to predict a feature map's shape before running it

2

You can now: build, train and evaluate a convolutional network that classifies images into three defect classes

3

You can now: recognise overfitting from training and validation curves and locate the epoch where generalisation stops improving

4

You can now: apply augmentation, dropout, weight decay and early stopping, and measure how much each narrows the overfitting gap

5

You can now: fine-tune a pre-trained network by freezing its backbone and replacing the classifier head

TOPIC 04

Vibe Coding Recurrent Networks for Sequence Data

Overview of RNNs, LSTM and GRU · Vibe coding an LSTM for time series forecasting · Tuning sequence models with follow-up prompts · Evaluating and visualizing model performance · Packaging a complete deep learning project

Key Concepts — Vibe Coding Recurrent Networks for Sequence Data

1

A recurrent network carries a hidden state from one timestep to the next, so order matters. A plain RNN forgets quickly because gradients vanish across long sequences.

2

An LSTM adds a cell state and three gates - forget, input, output - so it can retain information over hundreds of steps. A GRU merges the gates into a cheaper, faster cell.

3

Sequence data must be windowed before a model sees it: turn one long series into many (lookback -> horizon) pairs, and split by TIME, never at random.

4

Scale the series using statistics from the training window only. Fitting the scaler on the whole series leaks the future into the past and inflates every score.

5

`nn.LSTM` returns output and `(h_n, c_n)`, and `batch_first` changes the axis order. Reading the wrong axis is the classic silent sequence bug - it runs and predicts noise.

6

A forecast is judged against a naive baseline: predicting 'tomorrow equals today' is often strong. A model that cannot beat persistence has not learned the series.

Key Concepts — Vibe Coding Recurrent Networks for Sequence Data (cont.)

7

Packaging is what makes the work real: a clear folder structure, pinned dependencies, a README somebody else can follow, and a model card recording where the model must NOT be used.

Hands-On Labs — Vibe Coding Recurrent Networks for Sequence Data

Labs 17–18

- Overview of RNNs, LSTM and GRU
- Vibe Coding an LSTM for Time Series Forecasting

Labs 19–20

- Tuning Sequence Models with Follow-Up Prompts
- Evaluating and Visualizing Model Performance

Lab 21

- Packaging a Complete Deep Learning Project

Overview of RNNs, LSTM and GRU

LAB 17

Sequence models introduce a new axis and a new class of silent bug, so you explore the cells before you forecast with them. You prompt for a script that runs the same input batch through `nn.RNN`, `nn.LSTM` and `nn.GRU`, printing every output and hidden-state shape, the parameter count of each, and what LSTM returns that the others do not. You then demonstrate the vanishing-gradient problem directly: propagate a gradient back through a long sequence in an RNN and in an LSTM and compare the magnitudes at the first timestep. Finally you meet `batch_first`, the axis-order flag that silently trains models on nonsense.

You'll build

An `rnn_cells.py` comparing the three cells' shapes, gates and parameter counts, plus a measured vanishing-gradient demonstration

Tools: PyTorch `nn.RNN` / `nn.LSTM` / `nn.GRU`, Cursor / GitHub Copilot / Claude

Overview of RNNs, LSTM and GRU

STEP 1 OF 8

1

Prompt for the three-cell comparison. Insist on printing every returned object's shape, because that is where the confusion lives.

PROMPT — paste this into your AI coding assistant

Create `rnn_cells.py` comparing PyTorch's three recurrent cells.

SHAPES: the input is a batch of 16 sequences, each 50 timesteps long with 1 feature per step, so shape (16, 50, 1) with `batch_first=True`.

TASK: show exactly what each cell returns and how many parameters it has.

OPERATIONS: build `nn.RNN`, `nn.LSTM` and `nn.GRU` each with `input_size=1`, `hidden_size=32`, `num_layers=1`, `batch_first=True`. For each: run the batch through it; print the shape of EVERY returned object, naming them (output, and `h_n`, and for LSTM also `c_n`); print the total parameter count; and print a one-line description of its gates - RNN has none, GRU has reset and update, LSTM has forget, input and output.

Then print a comparison table of the three parameter counts and add a comment explaining why LSTM has roughly four times the parameters of RNN and GRU roughly three times.

CONSTRAINTS: manual seed 42, no training. Label every printed line clearly.

Overview of RNNs, LSTM and GRU

STEP 2 OF 8



2

Before running, predict two shapes: what shape is ``output`` and what shape is ``h_n``? They are different, and confusing them is the classic bug.

Overview of RNNs, LSTM and GRU

STEP 3 OF 8

3

Run it and check your predictions.

COMMAND — run this in your terminal

```
$ python rnn_cells.py
```

Overview of RNNs, LSTM and GRU

STEP 4 OF 8

4

Now demonstrate the `batch_first` trap, which is the reason sequence models silently learn nothing.

PROMPT — paste this into your AI coding assistant

Add a function `batch_first_trap()` to `rnn_cells.py`.

OPERATIONS: take the same (16, 50, 1) input. Run it through an `nn.LSTM` built with `batch_first=True` and one built with `batch_first=False`, WITHOUT transposing the input for the second. Print the output shape from each and the shape of `h_n` from each.

Add a comment explaining what the second model actually did: it interpreted the batch axis as the time axis, so it treated 16 timesteps of a 50-sequence batch. Note that this raises NO error and produces a plausibly shaped tensor, which is why it is so dangerous.

Then show the correct fix - transposing the input to (50, 16, 1) for the `batch_first=False` model - and confirm the output shapes now correspond.

Overview of RNNs, LSTM and GRU

STEP 5 OF 8

5

Run it and study what happened. Confirm for yourself that nothing raised an error.

COMMAND — run this in your terminal

```
$ python rnn_cells.py
```

Overview of RNNs, LSTM and GRU

STEP 6 OF 8

6

Measure the vanishing gradient rather than just describing it.

PROMPT — paste this into your AI coding assistant

Add a function `vanishing_gradient()` to `rnn_cells.py`.

TASK: measure how gradient magnitude decays back through time in an RNN versus an LSTM.

OPERATIONS: create an input of shape `(1, 100, 1)` with `requires_grad=True`. Pass it through a single-layer `nn.RNN(1, 16, batch_first=True)`, take the LAST timestep of the output, sum it, and call `backward()`. Record the absolute gradient magnitude at the input for each of the 100 timesteps. Repeat with an `nn.LSTM` of the same size. Plot both gradient-magnitude curves against timestep on a LOG y axis and save to `reports/gradient_flow.png`. Print the ratio of gradient magnitude at timestep 0 to timestep 99 for each cell.

CONSTRAINTS: same seed and hidden size for both so the comparison is fair.

Overview of RNNs, LSTM and GRU

STEP 7 OF 8

7

Run it, open reports/gradient_flow.png, and read the two ratios. This is the vanishing gradient, measured.

COMMAND — run this in your terminal

```
$ python rnn_cells.py
```


Overview of RNNs, LSTM and GRU

STEP 8 OF 8



8

Write your conclusion in prompts.md: which cell you will use in Lab 18, and the one-sentence reason.

Overview of RNNs, LSTM and GRU

✓ Test it

`rnn_cells.py` prints output as `torch.Size([16, 50, 32])` and `h_n` as `torch.Size([1, 16, 32])` for all three cells, plus `c_n` for the LSTM only, with parameter counts in roughly a 1:3:4 ratio. `batch_first_trap` shows a wrongly shaped-but-unraised result and its fix. `reports/gradient_flow.png` shows the RNN gradient decaying far faster than the LSTM's on a log axis.

Vibe Coding an LSTM for Time Series Forecasting

LAB 18

Forecast ForgeSight's vibration sensor. This lab contains two traps that no error message will ever reveal. First, sequence data must be split by TIME — a random split lets the model train on data from after the period it is tested on. Second, the scaler must be fitted on the training window only. You establish the persistence baseline before modelling, prompt for the windowing and the LSTM with both constraints stated explicitly, then verify the split and the scaler yourself before trusting any score. A forecaster that cannot beat 'tomorrow equals today' has not learned the series.

You'll build

An LSTM vibration forecaster with a time-ordered split and train-only scaling, measured against a persistence baseline

Tools: PyTorch nn.LSTM, pandas, engine.py, Cursor / GitHub Copilot / Claude

Vibe Coding an LSTM for Time Series Forecasting

STEP 1 OF 8

1

Look at the series before modelling it, and establish the persistence baseline.

PROMPT — paste this into your AI coding assistant

Create `baseline_series.py` for `data/vibration_series.csv`, which has a timestamp column and a `vibration_rms` column of roughly 3000 hourly readings.

OPERATIONS: (1) load it, print the row count, the date range and basic statistics; (2) plot the full series to `reports/series_overview.png`; (3) compute the PERSISTENCE baseline on the last 20% of the series - predicting that each value equals the previous value - and print its MAE and RMSE; (4) print the standard deviation of the series for comparison.

Add a comment explaining why persistence is the correct baseline for a forecasting task and what it means if a model cannot beat it.

Vibe Coding an LSTM for Time Series Forecasting

STEP 2 OF 8

2

Run it, open the plot, and write the persistence MAE down. This is the number your LSTM must beat.

COMMAND — run this in your terminal

```
$ python baseline_series.py
```

Vibe Coding an LSTM for Time Series Forecasting

STEP 3 OF 8

3

Prompt for the windowing, with the time-split constraint stated as the primary requirement.

PROMPT — paste this into your AI coding assistant

Create windowing.py that prepares data/vibration_series.csv for an LSTM.

TASK: turn one long univariate series into supervised (lookback -> horizon) training pairs.

OPERATIONS: build make_windows(series, lookback=48, horizon=1) returning X of shape (N, lookback, 1) and y of shape (N, 1). Then build get_series_loaders(lookback=48, batch_size=64) that:

1. Loads the series in TIME ORDER - never sorted or shuffled before splitting
2. Splits chronologically into the first 70% train, next 15% validation, final 15% test - by INDEX, not randomly
3. Fits the scaler using the TRAINING SEGMENT ONLY, computing mean and std from the training values, then applies those same values to validation and test
4. Windows each segment separately so no window ever spans a split boundary
5. Returns loaders plus the scaler statistics so predictions can be converted back to the original units

CONSTRAINTS: this is critical - do NOT use train_test_split with shuffle, and do NOT fit the scaler on the full series. Shuffling is allowed WITHIN the training loader only. Print the index ranges of the three splits and the training mean and std when run as a script.

EXPECTED OUTPUT: printed split ranges proving they are contiguous and non-overlapping, plus X and y shapes for each split.

Vibe Coding an LSTM for Time Series Forecasting

STEP 4 OF 8

4

Verify the two traps yourself. Check the printed split ranges are contiguous and in ascending order, and find the exact line where the scaler statistics are computed.

COMMAND — run this in your terminal

```
$ python windowing.py
```

Vibe Coding an LSTM for Time Series Forecasting

STEP 5 OF 8

5

Prompt for the model and training, reusing the engine again.

PROMPT — paste this into your AI coding assistant

Create lstm_forecast.py.

SHAPES: batches are X of (64, 48, 1) and y of (64, 1).

OPERATIONS: add class VibrationLSTM(nn.Module) to models.py with nn.LSTM(input_size=1, hidden_size=64, num_layers=2, batch_first=True, dropout=0.2) followed by nn.Linear(64, 1). In forward, take the LAST timestep of the LSTM output with output[:, -1, :] and pass it to the Linear layer - add a comment on why this is the right slice and what output[:, 0, :] would give instead.

Train with fit from engine.py: nn.MSELoss, Adam lr=1e-3, 60 epochs, an MAE metric computed in the ORIGINAL units by inverting the scaling.

CONSTRAINTS: batch_first=True must be explicit. Print the output shape of the first forward pass to prove the axis order.

EXPECTED OUTPUT: per-epoch losses, final test MAE and RMSE in original units, printed next to the persistence baseline MAE, and a forecast-versus-actual plot of the test segment saved to reports/forecast.png.

Vibe Coding an LSTM for Time Series Forecasting

STEP 6 OF 8



6

Train it and compare the MAE against your persistence baseline.

COMMAND — run this in your terminal

```
$ python lstm_forecast.py
```

Vibe Coding an LSTM for Time Series Forecasting

STEP 7 OF 8



Open reports/forecast.png and look at the shape of the prediction, not just the metric.

Vibe Coding an LSTM for Time Series Forecasting

STEP 8 OF 8

8

Prove to yourself that the time split mattered, by deliberately doing it wrong.

PROMPT — paste this into your AI coding assistant

Add a function `leaky_comparison()` to `lstm_forecast.py` that repeats the entire pipeline with two deliberate errors: a RANDOM shuffled split instead of a chronological one, and a scaler fitted on the FULL series before splitting. Train the identical model for the same 60 epochs and print the resulting test MAE next to the correct pipeline's MAE and the persistence baseline. Add a comment stating which number you would have reported if you had never checked the split, and why it is not achievable in production.

Vibe Coding an LSTM for Time Series Forecasting

✓ Test it

windowing.py prints three contiguous ascending index ranges and the scaler statistics computed from the training segment only. lstm_forecast.py prints the first forward pass shape as torch.Size([64, 1]), reports a test MAE in original units alongside the persistence baseline, and saves reports/forecast.png. leaky_comparison prints a better-but-unachievable MAE from the shuffled, fully-scaled pipeline.

Tuning Sequence Models with Follow-Up Prompts

LAB 19

Tuning is where vibe coding either pays off or descends into random prompting. You run a structured search over the choices that matter for a sequence model — lookback length, hidden size, layer count, cell type and learning rate — changing one thing at a time and recording every result in a table. The discipline being trained is the follow-up prompt: naming a specific symptom and a specific change rather than asking the assistant to 'make it better'. You finish with a tuned model, a results table that justifies it, and an explicit note of which changes made no difference at all.

You'll build

A tuning results table across lookback, hidden size, layers, cell type and learning rate, plus a justified best model

Tools: PyTorch, engine.py, Cursor / GitHub Copilot / Claude

Tuning Sequence Models with Follow-Up Prompts

STEP 1 OF 8

1

First, see what an undisciplined prompt produces, so the contrast is concrete.

PROMPT — paste this into your AI coding assistant

My LSTM forecaster gets a test MAE only slightly better than a persistence baseline. Make it better.

Tuning Sequence Models with Follow-Up Prompts

STEP 2 OF 8



2

Read what came back and count how many things it changed simultaneously. Do not run it — this is evidence, not code.

Tuning Sequence Models with Follow-Up Prompts

STEP 3 OF 8

3

Now the disciplined version. Prompt for a structured one-factor-at-a-time search.

PROMPT — paste this into your AI coding assistant

Create tune_lstm.py running a one-factor-at-a-time search over the vibration forecasting model.

BASELINE CONFIG: lookback=48, hidden_size=64, num_layers=2, cell=LSTM, lr=1e-3, 40 epochs.

RUNS: starting from that baseline, vary ONE factor at a time:

- lookback in [12, 24, 48, 96]
- hidden_size in [16, 32, 64, 128]
- num_layers in [1, 2, 3]
- cell in [LSTM, GRU]
- lr in [1e-2, 1e-3, 1e-4]

For each run record: the factor changed, its value, best validation MAE in original units, test MAE in original units, epochs to best, and wall-clock seconds.

CONSTRAINTS: reset manual seed 42 before every run and build a fresh model each time. Identical data and batch size across all runs. Include the persistence baseline MAE as a constant column so every row can be read against it.

EXPECTED OUTPUT: a printed table grouped by factor, saved to reports/tuning_results.csv, and a figure with one subplot per factor showing test MAE against the factor's value.

Tuning Sequence Models with Follow-Up Prompts

STEP 4 OF 8

4

Run the search. It is a few dozen short training runs — expect several minutes on CPU.

COMMAND — run this in your terminal

```
$ python tune_lstm.py
```

Tuning Sequence Models with Follow-Up Prompts

STEP 5 OF 8



5

Read the table by factor and answer: which factor produced the largest spread in test MAE, and which produced almost none?

Tuning Sequence Models with Follow-Up Prompts

STEP 6 OF 8

6

Practise the disciplined follow-up on whatever your table actually showed. Adapt this to your own result.

PROMPT — paste this into your AI coding assistant

In `tune_lstm.py` the lookback sweep shows test MAE improving from 12 to 48 timesteps but getting worse at 96, which suggests the longer window adds noise rather than signal. Add two intermediate values, 64 and 80, to the lookback sweep only. Change nothing else - same seed, same architecture, same number of epochs - and print the extended lookback results as a single sorted table so I can see where the minimum actually sits.

Tuning Sequence Models with Follow-Up Prompts

STEP 7 OF 8

7

Train the winning configuration properly and check it against the baseline one final time.

PROMPT — paste this into your AI coding assistant

Add a function `train_best()` to `tune_lstm.py` that takes the best configuration from `reports/tuning_results.csv`, trains it for 120 epochs with early stopping on validation loss, reports test MAE and RMSE in original units next to the persistence baseline, saves the model with `save_checkpoint` to `models/vibration_lstm_v1.pt` recording the full winning configuration in the metrics dict, and writes `models/vibration_model_card.md` stating the task, the lookback, the baseline comparison and the fact that the split was chronological.

Tuning Sequence Models with Follow-Up Prompts

STEP 8 OF 8



8

Record in prompts.md the three follow-up prompts that worked and, just as importantly, which tuned factors turned out not to matter.

Tuning Sequence Models with Follow-Up Prompts

Test it

reports/tuning_results.csv contains one row per configuration with the factor changed, both MAE columns and the persistence baseline as a constant column. You can name the factor with the largest spread and the one with the smallest. models/vibration_lstm_v1.pt holds the winning configuration and beats the persistence baseline, and prompts.md records three working follow-up prompts.

Evaluating and Visualizing Model Performance

LAB 20

One number cannot describe a model. You build the standard evaluation views for each of the three ForgeSight models — a parity plot and residual analysis for tool-wear regression, a normalised confusion matrix with per-class recall for defect classification, and forecast-versus-actual with error-by-horizon for vibration. Each view is chosen to answer a question the headline metric cannot: where is the model biased, which classes does it confuse, and how fast does the forecast decay. You assemble them into one report figure and write the honest summary that goes with it.

You'll build

A complete evaluation report covering all three models, with the plots and the written findings a stakeholder would need

Tools: PyTorch, matplotlib, scikit-learn metrics, Cursor / GitHub Copilot / Claude

Evaluating and Visualizing Model Performance

STEP 1 OF 8

1

Build the regression evaluation views for the tool-wear model.

PROMPT — paste this into your AI coding assistant

Create evaluate.py with a function eval_regression() for the tool-wear model.

OPERATIONS: load models/wear_net_v1.pt (or retrain briefly if it was not saved), predict on the test split, and produce a 3-panel figure saved to reports/eval_wear.png:

- (1) a parity plot of predicted against actual with a $y=x$ reference line and the R-squared value in the title;
- (2) a residual plot of (predicted - actual) against actual, with a horizontal zero line;
- (3) a histogram of residuals with the mean and standard deviation printed in the title.

Also print test MAE, RMSE and R-squared next to the mean-predictor baseline.

Add a comment on what a residual plot that fans out or trends with the actual value tells you about the model.

Evaluating and Visualizing Model Performance

STEP 2 OF 8

2

Run it and read panel 2 specifically. Are the residuals centred on zero across the whole range, or does the model drift at high wear values?

COMMAND — run this in your terminal

```
$ python evaluate.py
```

Evaluating and Visualizing Model Performance

STEP 3 OF 8

3

Build the classification evaluation views.

PROMPT — paste this into your AI coding assistant

Add `eval_classification()` to `evaluate.py` for the defect CNN.

OPERATIONS: load the best defect model, predict on the validation set, and produce a 3-panel figure saved to `reports/eval_defects.png`:

(1) a confusion matrix normalised BY ROW so each cell shows the share of that true class, with class names on both axes and values annotated;

(2) a horizontal bar chart of per-class precision, recall and F1;

(3) a histogram of the model's maximum softmax probability, split into correct and incorrect predictions.

Print the classification report and the macro-averaged F1 next to the majority-class baseline accuracy.

Add a comment explaining why a row-normalised confusion matrix is more readable than raw counts when classes are imbalanced, and note that softmax is applied HERE for reporting, not in the model.

Evaluating and Visualizing Model Performance

STEP 4 OF 8



4

Run it and read panel 3. Is the model well calibrated — are its wrong answers given with lower confidence than its right ones?

COMMAND — run this in your terminal

```
$ python evaluate.py
```

Evaluating and Visualizing Model Performance

STEP 5 OF 8

5

Build the forecasting evaluation views, including the check for the model that only learned persistence.

PROMPT — paste this into your AI coding assistant

Add `eval_forecast()` to `evaluate.py` for the vibration LSTM.

OPERATIONS: load `models/vibration_lstm_v1.pt`, predict on the test segment, and produce a 3-panel figure saved to `reports/eval_forecast.png`:

- (1) forecast against actual over the test period in ORIGINAL units, with the persistence baseline as a third line;
- (2) a residual-over-time plot showing whether errors grow towards the end of the period;
- (3) a scatter of predicted change against actual change from one timestep to the next - this reveals whether the model is anticipating movement or merely repeating the last value.

Print test MAE and RMSE against the persistence baseline, plus the correlation between predicted and actual CHANGE.

Add a comment explaining that a model whose predicted change correlates near zero with actual change has learned persistence and nothing more.

Evaluating and Visualizing Model Performance

STEP 6 OF 8

6

Run it and read panel 3 and the change-correlation figure. This is the honest test of whether your forecaster learned anything.

COMMAND — run this in your terminal

```
$ python evaluate.py
```

Evaluating and Visualizing Model Performance

STEP 7 OF 8



Assemble the single report figure and write the findings.

PROMPT — paste this into your AI coding assistant

Add `build_report()` to `evaluate.py` that assembles one combined figure saved to `reports/evaluation_report.png` with a row per model - regression, classification, forecasting - showing the single most informative panel from each, with a clear row label and a shared title naming the course project and the date. Then write `reports/findings.md` with one short section per model containing: the headline metric next to its baseline, the most important thing the plots reveal that the metric does not, and one stated limitation of the model.

Evaluating and Visualizing Model Performance

STEP 8 OF 8



Read your own findings.md aloud as if presenting to a plant manager. If any sentence would not survive a follow-up question, rewrite it.

Evaluating and Visualizing Model Performance

Test it

reports/eval_wear.png, reports/eval_defects.png and reports/eval_forecast.png each show three labelled panels. reports/evaluation_report.png combines one panel per model with a title and date. reports/findings.md has three sections, each naming a metric with its baseline, an insight the plot revealed that the metric did not, and a stated limitation.

Packaging a Complete Deep Learning Project

LAB 21

Turn twenty labs of scripts into something you could hand to a colleague and walk away from. You restructure the workspace into a clean layout, pin the dependencies, write a `predict.py` command-line tool that loads any of the three saved models and scores new input, add a smoke test that proves the prediction path works end to end, and write the README and model cards. The real test is the last step: a partner clones your project into a fresh folder and runs it from the README alone, without asking you a single question.

You'll build

A packaged, documented, tested ForgeSight project with a working CLI that another person can run unaided

Tools: Python packaging, pytest, git, Cursor / GitHub Copilot / Claude

Packaging a Complete Deep Learning Project

STEP 1 OF 9

1

Restructure the workspace. Ask for a plan first rather than letting the assistant move files immediately.

PROMPT — paste this into your AI coding assistant

Review my torch-vibe project folder and propose a clean package structure for handover. Do NOT move anything yet - give me the proposed layout as a tree with a one-line purpose for each folder and each top-level file.

The project contains: data loading for tabular and image and series data; three trained models (tool-wear regression, defect CNN, vibration LSTM); a shared training engine; a checkpoint helper; evaluation scripts; saved checkpoints; and generated reports.

Constraints: source code separated from data, models and reports; every model loadable without running any training script; no absolute paths anywhere; data and model binaries excluded from version control but their absence explained in the README.

Packaging a Complete Deep Learning Project

STEP 2 OF 9



2

Review the proposed tree and adjust it before anything moves. You are the architect here, not the assistant.

Packaging a Complete Deep Learning Project

STEP 3 OF 9

3

Apply the restructure and fix the imports it breaks.

PROMPT — paste this into your AI coding assistant

Apply the agreed structure. Move the files, update every import to match, and confirm each of the three training scripts and the evaluation script still run from the project root. Replace any absolute path with a path relative to the project root resolved via pathlib. Report every file you moved and every import you changed.

Packaging a Complete Deep Learning Project

STEP 4 OF 9

4

Write the prediction CLI — the thing that makes the project usable by someone who will never read your training code.

PROMPT — paste this into your AI coding assistant

Create predict.py, a command-line tool for the packaged project.

USAGE: `python predict.py --model wear --input data/sample\_reading.csv`, `python predict.py --model defect --input path/to/image.png`, `python predict.py --model vibration --input data/recent\_series.csv`.

OPERATIONS: for each model - load the checkpoint, apply the SAME preprocessing the model was trained with by reading the saved statistics from the checkpoint, run inference under `torch.no_grad()` and `model.eval()`, and print a human-readable result. For the defect model print the predicted class name and its probability; for wear print the predicted microns; for vibration print the next predicted value and the persistence value alongside it for comparison.

CONSTRAINTS: validate the input file exists and has the expected columns or format, and fail with a clear message naming what is wrong rather than a traceback. Never re-fit any scaler - always use the statistics stored in the checkpoint. Include `--help` text for every argument.

EXPECTED OUTPUT: three working commands, each printing a labelled prediction.

Packaging a Complete Deep Learning Project

STEP 5 OF 9

5

Test all three commands yourself with the sample inputs.

COMMAND — run this in your terminal

```
$ python predict.py --model defect --input data/defects/val/scratch/scratch_001.png  
$ python predict.py --model wear --input data/sample_reading.csv  
$ python predict.py --model vibration --input data/vibration_series.csv
```

Packaging a Complete Deep Learning Project

STEP 6 OF 9

6

Add the smoke test that proves the prediction path works, so a future change that breaks it fails loudly.

PROMPT — paste this into your AI coding assistant

Create tests/test_predict.py using pytest.

TESTS: (1) each of the three checkpoints loads and returns a model in eval mode; (2) each model produces an output of the expected shape for a single synthetic input; (3) the defect model's softmax probabilities sum to 1 and the predicted class is a valid index; (4) predict.py exits with a clear non-zero status and a readable message for a missing input file; (5) the wear model's prediction changes when the input changes, which catches a model that always returns the same value.

CONSTRAINTS: tests must run without retraining anything and must not require internet access. Skip cleanly with a clear message if a checkpoint file is absent.

Packaging a Complete Deep Learning Project

STEP 7 OF 9

7

Run the tests and confirm they pass.

COMMAND — run this in your terminal

```
$ python -m pytest tests/ -v
```


Packaging a Complete Deep Learning Project

STEP 8 OF 9

8

Write the README and finalise the model cards, then pin the dependencies.

PROMPT — paste this into your AI coding assistant

Write README.md for the packaged project covering: what ForgeSight does and the three models it contains; the exact setup commands from a fresh clone including creating the virtual environment and installing requirements; how to obtain or regenerate the data, since data files are not in version control; how to run each of the three predictions with a copyable example command; how to retrain each model; the project structure as a tree; and a results table of each model's headline metric next to its baseline. Also create .gitignore excluding .venv, __pycache__, data/, models/*.pt and reports/. Then regenerate requirements.txt with pinned versions.

Packaging a Complete Deep Learning Project

STEP 9 OF 9



Run the handover test, which is the only verification that counts.

Packaging a Complete Deep Learning Project

Test it

A partner clones your project into a fresh folder, follows README.md alone, creates the environment, installs from requirements.txt and successfully runs all three ``python predict.py`` commands without asking you anything. ``python -m pytest tests/ -v`` passes. No absolute path appears anywhere in the source, and each model has a model card stating its metric, its baseline and where it must not be used.

Recap — Vibe Coding Recurrent Networks for Sequence Data

1

You can now: compare RNN, LSTM and GRU cells by their shapes, gates and parameter counts, and demonstrate why plain RNNs forget

2

You can now: window and split a time series correctly and train an LSTM that beats a persistence baseline

3

You can now: improve a sequence model through disciplined follow-up prompts and record what each change actually bought

4

You can now: build the evaluation views that reveal what a single metric hides, across all three ForgeSight models

5

You can now: package models, code, dependencies and documentation into a project another person can run



WRAP-UP

Course Summary & Next Steps

What You Achieved

1

Vibe coding setup

Explain what AI vibe coding is, set up Cursor, GitHub Copilot and Claude as PyTorch coding partners, and apply prompting patterns that produce correct deep learning code.

2

Tensors and autograd

Vibe code PyTorch tensor operations, computation graphs and autograd, and review and debug AI-generated PyTorch code rather than trusting it.

3

Neural networks

Build regression and classification neural networks in PyTorch from prompts, choosing architectures, activation functions and loss functions deliberately.

4

Train and iterate

Generate training loops, optimizers and metrics from prompts, and save, load and iterate on trained models.

5

CNNs for vision

Vibe code convolutional neural networks for image classification, diagnose overfitting, and apply data augmentation, regularization and transfer learning.

6

Sequences and shipping

Vibe code recurrent networks for sequence data, tune and evaluate them with follow-up prompts, and package a complete deep learning project.

Continuing Your Journey

1

Rebuild the ForgeSight project from an empty folder, prompting your assistant from scratch — that is the real test of the skill.

2

Replace the synthetic telemetry, images and vibration series with data from your own organisation and see which of the twenty-one steps breaks first.

3

Try the same labs in a different assistant (Cursor vs Copilot vs Claude) and compare the PyTorch each one produces.

4

Move on to computer vision, natural language processing or MLOps at www.tertiarycourses.com.sg.

Keep Practising After the Course

1

Re-run the labs on your own machine — repetition is what makes the skills stick.

2

Apply the techniques to a real process or project in your own organisation.

3

Your course material stays available at <https://lms-tms.tertiaryinfotech.com/>.

4

Explore the follow-on courses at www.tertiarycourses.com.sg.

THANK YOU FOR ATTENDING

Thank You!

You now have a complete, tested deep learning project and the prompting habits that let you build the next one without us.